

Unit 3 Analysis

Software Analysis & Design | CSIT 5th

Instructor: Pratiksha Poudel

Syllabus

Determining System Requirements: Introduction; Performing Requirements

Determination; Traditional Methods for Determining Requirements; Contemporary methods for determining System Requirements; Radical Methods for determining System Requirements.

Structuring System Process Requirements: Introduction; Process Modelling; Data Flow Diagrams; Modeling Logic with Decision Tables; Decision trees; and Pseudocodes

Structuring System Data Requirements: Introduction; Conceptual Data Modeling; Gathering Information for Conceptual Data Modeling; Introduction to E-R Modeling

Analysis

An analysis is the first system development life cycle (SDLC) phase where we begin to understand, in depth the need for system changes. An analysis is a large process, we divide it into two main activities to make it easier to understand:

1. Requirements Determination
2. Requirements Structuring

Requirements Determination

- A **requirement** is a statement about an intended product that specifies what it should do or how to do it. For requirements to be effectively implemented and measured, they must be specific, unambiguous and clear. For example, a requirement may be that a specific button must enable printing of the contents of the current screen.
- The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from the client's point of view.
- The process to gather the software requirements from the client, and analyze and document them is known as requirement engineering.

Software Requirements Characteristics

- Gathering software requirements is the foundation of the entire software development project. Hence they must be clear, correct and well-defined.
- A complete Software Requirement Specifications must be:
 - Clear
 - Correct
 - Consistent
 - Coherent
 - Comprehensible
 - Modifiable
 - Verifiable
 - Prioritized
 - Unambiguous
 - Traceable
 - Credible source

Types of Requirements

- Requirements are categorized in several ways. The following are common categorizations of requirements that relate to technical management:

Customer Requirements

Customer requirements refer to the specifications or features of a product or service that are deemed necessary by customers. These requirements motivate customers to buy a product or service. To determine customer requirements, companies can research their target market to understand their desires and needs. They can also take a more direct approach by asking their customers for feedback through polls, surveys or social media.

Types of Requirements

Functional Requirements

- Requirements, which are related to functional aspect of software fall into this category.
- They define functions and functionality within and from the software system.
- **Examples -**
 - Search option given to user to search from various invoices.
 - User should be able to mail any report to management.
 - Users can be divided into groups and groups can be given separate rights.
 - Should comply business rules and administrative functions.
 - Software is developed keeping downward compatibility intact.

Types of Requirements

Non-Functional Requirements

- Requirements, which are not related to functional aspect of software, fall into this category. They are implicit or expected characteristics of software, which users make assumption of.
- **Non-functional requirements include -**
 - Security
 - Logging
 - Storage
 - Configuration
 - Performance
 - Cost
 - Interoperability
 - Flexibility
 - Disaster recovery
 - Accessibility

Types of Requirements

User Interface requirements

- UI is an important part of any software or hardware or hybrid system. A software is widely accepted if it is -
 - easy to operate
 - quick in response
 - effectively handling operational errors
 - providing simple yet consistent user interface
- User acceptance majorly depends upon how user can use the software. UI is the only way for users to perceive the system. A well performing software system must also be equipped with attractive, clear, consistent and responsive user interface. Otherwise the functionalities of software system can not be used in convenient way. A system is said be good if it provides means to use it efficiently.

Types of Requirements

- User interface requirements are briefly mentioned below -
 - Content presentation
 - Easy Navigation
 - Simple interface
 - Responsive
 - Consistent UI elements
 - Feedback mechanism
 - Default settings
 - Purposeful layout
 - Strategical use of color and texture.
 - Provide help information
 - User centric approach
 - Group based view settings.

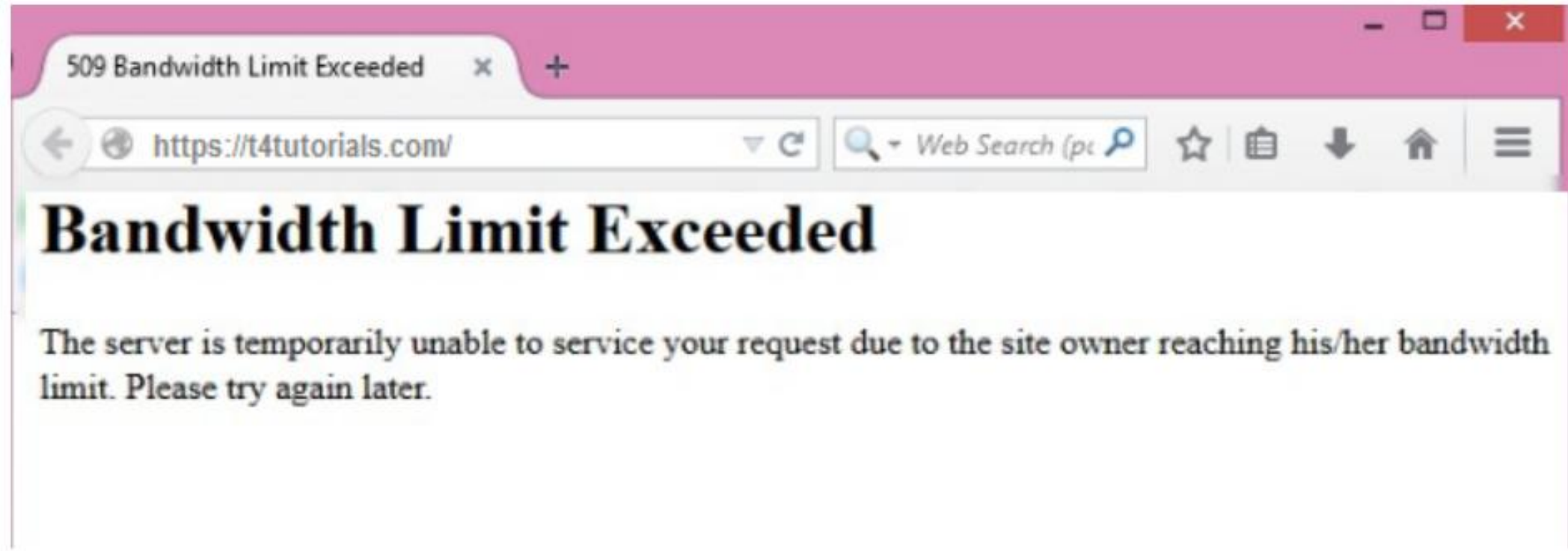
Types of Requirements

Performance Requirements

- Customers not only require that a product or service performs its intended purpose but also that it does it well. The definition of good performance varies, but examples may include its speed or accuracy when conducting tasks. Customers also purchase products specifically to improve the performance of another product. For example, a person who plays video games on their computer can buy a new graphics card to enable higher video quality.
- **Examples of Software Performance Requirements**
 - Response Time
 - Workload:
 - Scalability:
 - Platform:

Types of Requirements

Performance Requirement- Bandwidth exceeded



Creating a Requirements Definition

- Determine the types of functional and non-functional requirements applicable to the project
- Use requirements-gathering techniques to collect details
- Analysts work with users to verify, change and prioritize each requirement
- Continue this process through analysis workflow, but be careful of scope creep
- Requirements that meet a need but are not within the current scope can be added to a list of future enhancements

Sample of Requirements Definition

Nonfunctional Requirements

1. Operational Requirements

- 1.1. The system will operate in Windows environment.
- 1.2. The system should be able to connect to printers wirelessly.
- 1.3. The system should automatically back up at the end of each day.

2. Performance Requirements

- 2.1. The system will store a new appointment in 2 seconds or less.
- 2.2. The system will retrieve the daily appointment schedule in 2 seconds or less.

3. Security Requirements

- 3.1. Only doctors can set their availability.
- 3.2. Only a manager can produce a schedule.

4. Cultural and Political Requirements

- 4.1. No special cultural and political requirements are anticipated.

Functional Requirements

1. Manage Appointments

- 1.1. Patient makes new appointment.
- 1.2. Patient changes appointment.
- 1.3. Patient cancels appointment.

2. Produce Schedule

- 2.1. Office Manager checks daily schedule.
- 2.2. Office Manager prints daily schedule.

3. Record Doctor Availability

- 3.1. Doctor updates schedule

The Process of Determining Requirements

- Good Systems Analyst Characteristics:
 - Impertinence—Ask questions about everything that exists and also what may exist in the future.
 - Impartiality—Find the best solution to a business problem or opportunity.
 - Relaxing constraints—Eliminate unfeasibility, and assume anything is possible.
 - Attention to detail—Everything must fit together so that the system works properly.
 - Reframing—Every system is different and needs a new creative approach.

Deliverables and Outcomes for Requirements Determination:

The primary deliverables from requirements determination are various forms of information gathered during the determination process: transcripts of interviews, notes from observations, reports, job descriptions, analyzed responses from questionnaires and sets of forms.

- From interviews and observations — interview transcripts, observation notes, meeting minutes
- From existing written documents — mission and strategy statements, business forms, procedure manuals, job descriptions, training manuals, system documentation, flowcharts
- From computerized sources — Joint Application Design session results, CASE repositories, reports from existing systems, displays, and reports from system prototype

Methods for Determining System Requirements

1. Traditional Methods
2. Contemporary Methods
3. Radical Methods

1. Traditional Methods for Determining Requirements

1.1 Interviewing

- individuals
- groups

1.2 Observing workers

1.3 Studying business documents

1.1- Interviewing and Listening

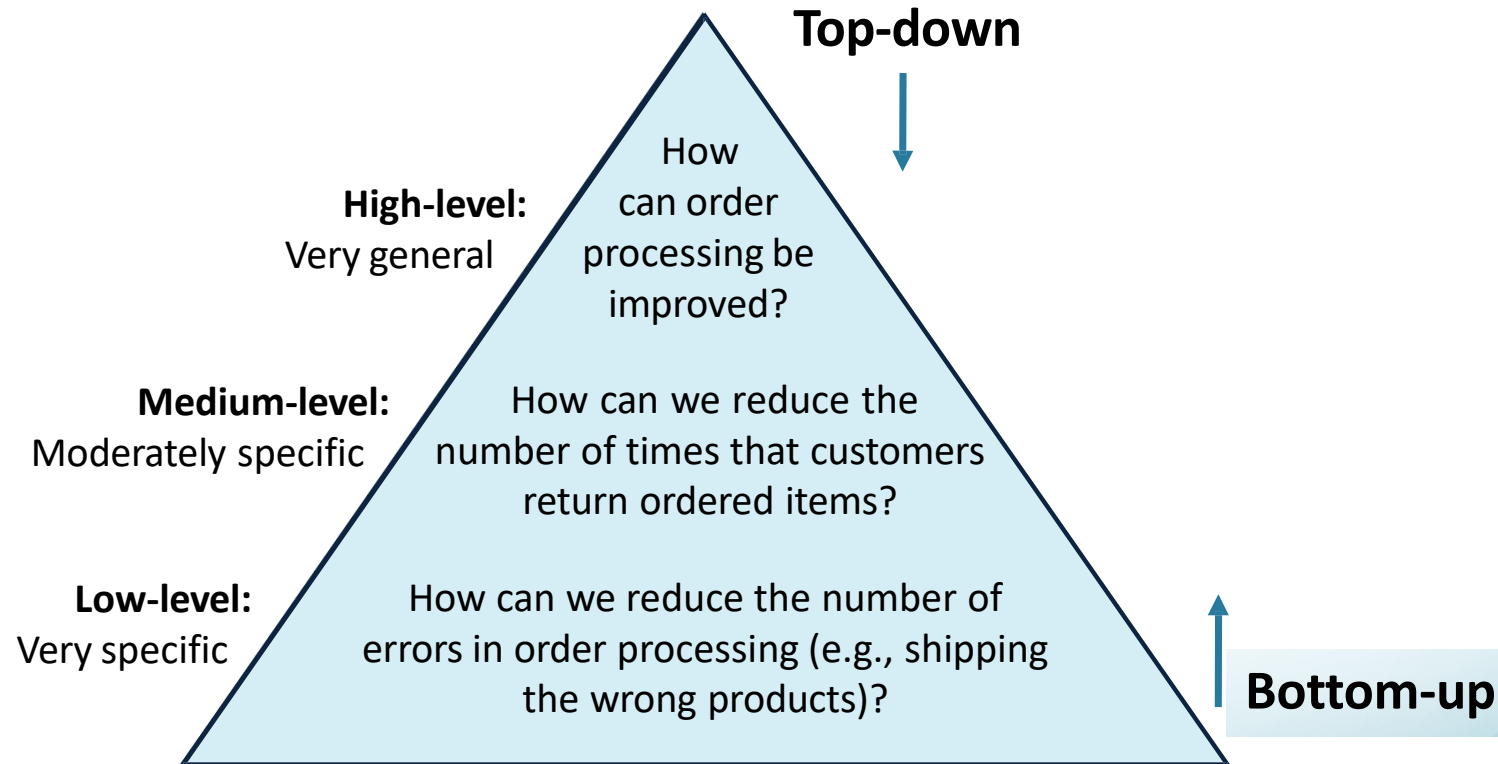
- The personal **interview** is generally recognized as the most often used fact-finding technique. Interviews are the fact-finding techniques whereby the systems analysts collect information from individuals through face-to-face interaction.
- There are two roles assumed in an interview. The systems analyst is the **interviewer**, responsible for organizing and conducting the interview. The system user or system owner is the **interviewee**, who is asked to respond to a series of questions.

Interviews

Types of Interviews

- There are two types of interviews: **unstructured** and **structured**.
- **Unstructured interviews** are conducted with only a general goal or subject in mind and with few, if any, specific questions.
- **Structured interviews** on the other hand are conducted with a set of specific questions to ask the interviewee.

Interviewing Strategies



Types of Interview Questions

Types of Questions

- Open-ended questions are usually used to probe for information for which you cannot anticipate all possible responses or for which you do not know the precise question to ask.
- Closed-ended questions provide a range of possible answers from which the interviewee may choose (True/False, Multiple Choice, Ratings a response, Ranking items, etc.)

Interview Guidelines:

- Do not phrase a question in a way that implies a right or wrong answer.
- Do not ask questions that imply YES (TRUE) or NO (FALSE) answers.
- Listen very carefully to what is being said, and take careful notes.
- Once the interview is over, go back to your office and type up your notes within 24-48 hours.
- Be careful during the interview not to set expectations about the new or replacement system unless you are sure these features will be part of the delivered system.

Interview Guidelines:

- **Advantages**

- Interviews give the analyst an opportunity to motivate the interviewee to respond freely and openly to questions.
- Interviews allow more feedback from the interviewee.
- Interviews give the analyst an opportunity to observe the interviewee's nonverbal communication.

- **Disadvantages**

- Interviewing is a very time-consuming and therefore costly fact-finding approach.
- Success of interviews is highly dependent on the systems analyst's human relational skills.
- Interviewing may be impractical due to the location of interviewees.

1.2- Interviewing Groups

- Group interviews are similar to one-on-one interview, except there is more than one person being interviewed. Group interviews work well when the interviewees are at the same level or position.
- A group interview also has an advantage when there is a time constraint. More thoughts and discussion can be generated, as someone in the group may state or suggest an idea that may have been overlooked by others, which in turn can lead to a discussion or provide more information on a particular issue.

Advantages

- More effective use of time
- Can hear agreements and disagreements at once
- Opportunity for synergies

• Disadvantages

- More difficult to schedule than individual interviews

Nominal Group Technique (NGT)

- The **Nominal Group Technique** is a form of brainstorming, wherein a structured meeting is held among the group members where they are required to find solutions to the problem identified for the discussion.
- a systematic and an organized group meeting held among the members to facilitate decision making by properly identifying the problems and generating the solutions thereof.

Questionnaires

- This technique is used to conduct surveys through questionnaires. **Questionnaires** are special purpose documents that allow the analyst to collect information and opinions from the respondents. The document can be mass -produced and distributed to respondents, who can then complete the questionnaire on their own time. Questionnaires allow the analyst to collect facts from a large number of people.
- **Types of Questionnaires**
- There are two formats of questionnaire, **free-format** and **fixed-format**. Free-format questionnaire offer the respondent to record the answer in the space provided after the questionnaire.

Questionnaires

Fixed-format questionnaire contain questions that require selection of predefined responses. In this format, the respondent must choose from the available answer. There are three types of fixed-format questions:

Multiple Choice Questions: - The respondent is given several answers of a question. The respondent should be told if more than one answer can be selected.

Rating Questions: - The respondent is given a statement and asked to use supplied responses to state an opinion.

Ranking questions: - The respondent is given several possible answers, which are to be ranked in the order of preference or experience.

Questionnaires

Advantages

- Most questionnaires can be answered quickly.
- Inexpensive for gathering data from a large number of individuals.
- Responses can be tabulated and analyzed quickly.

Disadvantages

- There is no guarantee that an individual will answer or expand on all the questions.
- Questionnaires tend to be inflexible.
- It not possible for the systems analyst to observe and analyze the respondent's body language.
- There is no immediate opportunity to clarify a vague or incomplete answer to questions.
- Good questionnaires are difficult to prepare.
- The number of respondents is often low.

1.3- Directly Observing Users

- Observation is an effective data-collection technique for obtaining an understanding of a system. In this technique, the systems analyst either participates in or watches a person performing activities to learn about the system.

Advantages:

- Data gathered can be highly reliable. Sometimes observations can be conducted to check the validity of data obtained directly from individuals.
- The systems analyst is able to see exactly what is being done.
- It is relatively inexpensive compared to other fact-finding techniques, because other techniques usually require more employees.
- It allows the system analyst to do work measurement.

1.3- Directly Observing Users

Disadvantages:

- People usually feel uncomfortable when being watched to their work.
- Some system activities may take place at odd times, causing a scheduling inconvenience for the system analyst.
- It may cause interruption.
- Some tasks may not always be performed by observation.

1.4- Analyzing Procedures and Other Documents

- **Document Analysis**

- Review of existing business documents
- Can give a historical and “formal” view of system requirements

- **Types of Documents**

- Written work procedure
- Business form
- Report
- Description of current information system

1.4- Analyzing Procedures and Other Documents

- Analyzing existing documents can prove to be a useful technique in requirement gathering, on its own as well using it to supplement other techniques.
- Reviewing the current process and documentation can help the analyst understand the business, or system, and its current situation. Existing documentation will provide the analyst the titles and names of stakeholders who are involved with the system.
- The analyst may find there was missing information in old documents. They may also find redundancy, in which steps are unnecessarily repeated.
- The consultant may look at old requirement documents and reuse of the requirements that may still be relevant, while discarding others that may be out of date. The reason why the current system is designed the way it is, which can suggest why certain features were left out.

Analyzing Procedures and Other Documents ctd....

- Analyzing documents can be used as a supplement to information obtained from interviews, questionnaires, and observations. For example, if some of the interview answers are unclear, organizational documents may help in making sense of some of the interviewee's answers.
- Reviewing existing documents may also assist in understanding why a user performs certain tasks while observing them.
- A drawback to analyzing documents is that documents may be outdated, the analyst must confirm whether the documents are current or not. Another drawback to reviewing documents is it can be very time consuming, depending on the organization and the system.
- A system that interacts with many different facets of the business will have large amounts of documentation to review

Analyzing Procedures and Other Documents (Contd...)

- **Types of information to be discovered:**
 - Problems with existing system
 - Opportunity to meet new need
 - Organizational direction
 - Names of key individuals
 - Values of organization
 - Special information processing circumstances
 - Reasons for current system design
 - Rules for processing data

2- Contemporary Methods for Determining System Requirements

- **Joint Application Design (JAD)**

- Brings together key users, managers, and systems analysts
- Purpose: collect system requirements simultaneously from key people
- Conducted off-site

- **Group Support Systems**

- Facilitate sharing of ideas and voicing of opinions about system requirements

Contemporary Methods for Determining System Requirements (Cont.)

- **CASE tools**

- Used to analyze existing systems
- Help discover requirements to meet changing business conditions

- **System prototypes**

- Iterative development process
- primary working version of system
- Refine understanding of system requirements in concrete terms

2.1 Joint Application Design (JAD)

- JAD is an example of a more contemporary method for gathering requirements. Joint Application Development (JAD) was introduced in the late 1970s to solve some of the problems users experienced in the conventional methods used to gather requirements.
- Organizations that have adopted JAD report savings of 40% or more in project design time. JAD is successful in reducing time because the user and other key members are heavily involved in the process.
- The goal is to get the design right the first time, thereby reducing different iterations.
- JAD is usually conducted in a location other than the place where the people involved work. This helps to reduce distractions.
- The participants of a JAD include: JAD session leader (also known as the facilitator), users, managers, sponsors, systems analysts, scribe, and other IS staff members. The facilitator, who is usually trained in project management as well as systems analysis, manages the entire process.
- They are responsible for keeping the group on the agenda and resolving conflicts.

Joint Application Design (JAD) (contd...)

- Facilitators do not contribute ideas; they solicit the ideas of the group. They also make the rules for the sessions, and can dismiss anyone who is troublesome to the process.
- Users involvement helps eliminate their frustration over the systems development process. The user is involved throughout a JAD session, and can make great contributions, as they will be the ones using the new system.
- Managers can provide more of insight into the organizational aspects of the system.
- The sponsor doesn't attend all JAD sessions, but usually just the beginning or end, as they do not contribute to ideas, they merely sponsor the process.
- Systems analysts are there to learn from users and managers. As in other requirements gathering techniques, they should not show bias. The scribe takes detailed notes during the session. The other IS staff members, such as programmers or database analysts attend to contribute technical aspects to the proposed ideas.

Joint Application Design (JAD)

- It is a process whereby highly structured group meeting is conducted to analyze problems and define requirements. JRP (Joint Requirement Planning) is a subset of a more comprehensive ***joint application development (JAD)***. The JRP participants are:
 - **Sponsor:** - This person is normally an individual who is in top management and has authority that spans the different departments and users who are to be involved in the systems project.
 - **Facilitator:** - The JRP facilitator or leader is usually responsible for leading all sessions that are held for a systems project.
 - **Users and Managers:** - Users devote themselves to the JRP sessions to effectively communicate business rules and requirements, review design prototypes and make acceptance decisions. Managers approve project objectives, establish project priorities, approve schedules and costs, and approve identified training needs and implementation plans.
 - **Scribe(s):** - Scribes are responsible for keeping records pertaining to everything discussed in the meeting.
 - **IT Staff:** - IT personnel listen and take notes regarding issues and requirements voiced by the users and managers. Normally, IT personnel do not speak unless invited to do so.

JAD (Contd...)

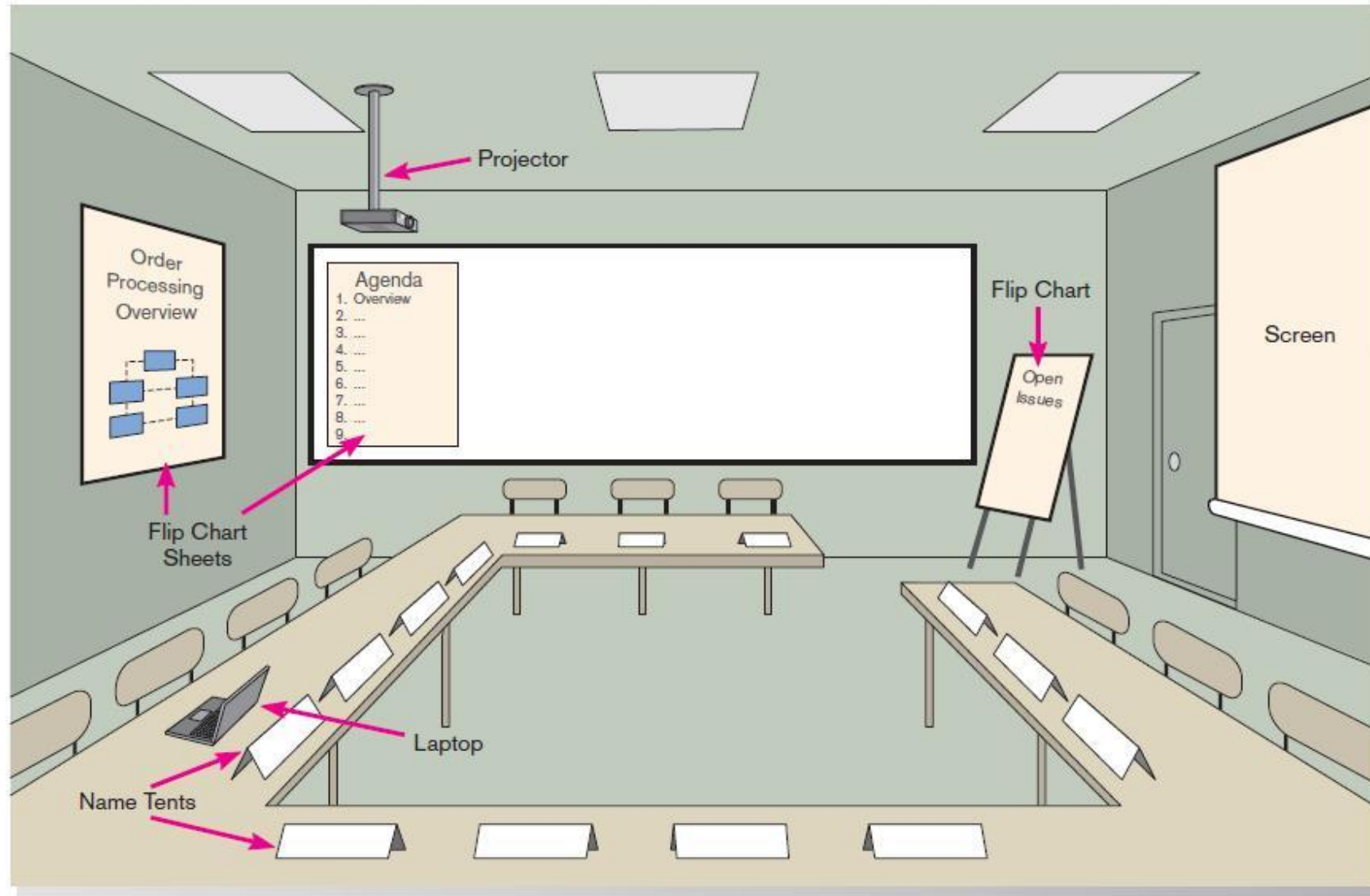


FIGURE: Illustration of the typical room layout for a JAD

Source: Based on Wood and Silver, 1995

JAD (Contd...)

Benefits

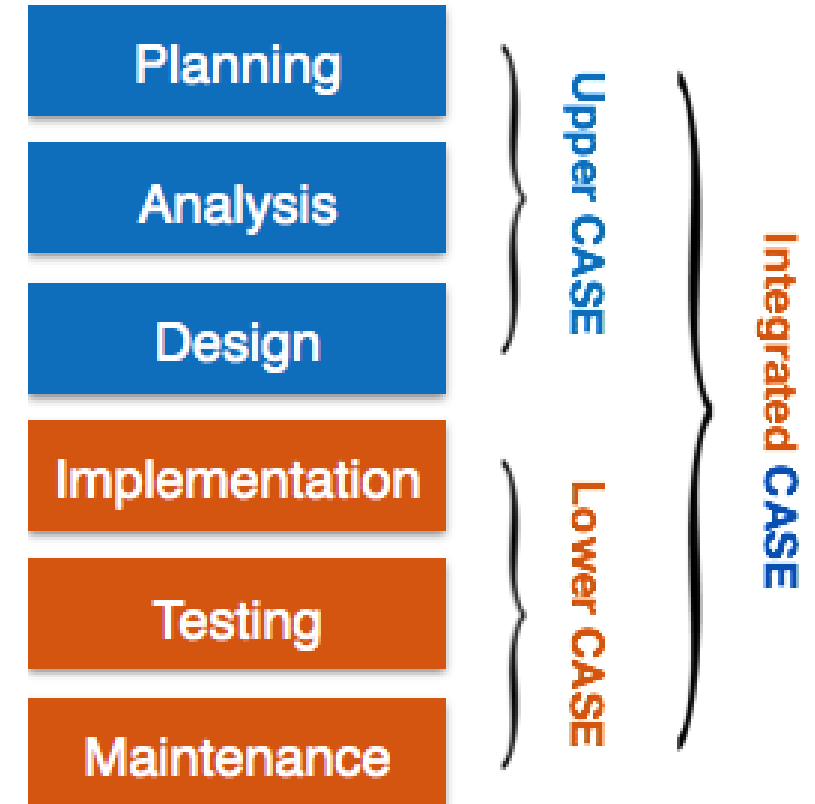
- It actively involves users and managers in the development project.
- It reduces the amount of time required to develop systems.
- When JRP incorporates prototyping as a means for conforming requirements and obtaining design approvals, the benefits of prototyping are realized.

2.2- CASE Tools

- There are number of CASE tools available to simplify various stages of Software Development Life Cycle such as Analysis tools, Design tools, Project management tools, Database Management tools, Documentation tools are to name a few.
- CASE analysis tools help to gather requirements, automatically check for any inconsistency, inaccuracy in the diagrams, data redundancies or erroneous omissions. For example, Accept 360, Accompa, CaseComplete for requirement analysis, Visible Analyst for total analysis.

2.2- CASE Tools

- **Upper Case Tools** - Upper CASE tools are used in planning, analysis and design stages of SDLC.
- **Lower Case Tools** - Lower CASE tools are used in implementation, testing and maintenance.
- **Integrated Case Tools** - Integrated CASE tools are helpful in all the stages of SDLC, from Requirement gathering to Testing and documentation.



2.2- CASE Tools

Best Requirements Management Tools for Data Gathering

- [Modern Requirements](#)
- [Jama Software](#)
- [Visure](#)
- [SpiraTeam](#)

2.3- Using Prototyping During Requirements Determination

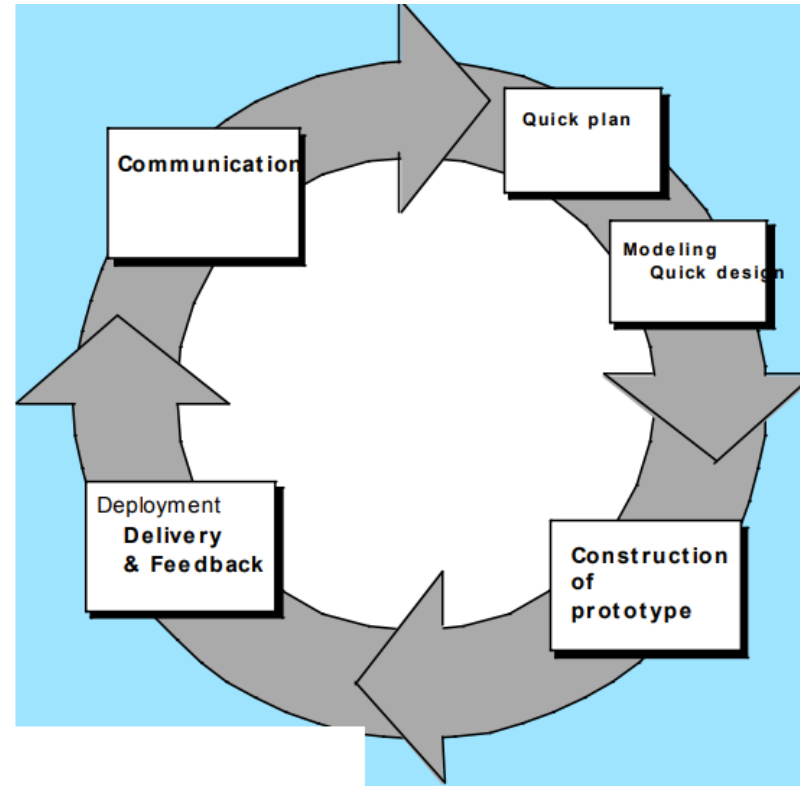
- Prototyping is a means of exploring ideas before you invest in them.
- Prototyping can be used during requirements determination to collect user requirements and present them in the form of a working system prototype.
- Quickly converts requirements to working version of system
- Users can look at, play with, and compare the prototype to their system requirements. Analysts can then adjust the prototype to better fit what the users have in mind.
- Prototyping allow system analysts quickly show users the basic requirement into a working version of the desired information system. After viewing and testing the prototype, the users usually adjust existing requirements to new ones.

Using Prototyping During Requirements Determination (Contd...)

- **Most useful when:**

- User requests are not clear.
- Few users are involved in the system.
- Designs are complex and require concrete form.
- There is a history of communication problems between analysts and users.
- Tools are readily available to build prototype.

2.3- Using Prototyping During Requirements Determination



- **Drawbacks**

- Tendency to avoid formal documentation
- Difficult to adapt to more general user audience
- Sharing data with other systems is often not considered
- Systems Development Life Cycle (SDLC) checks are often bypassed

Using Prototyping During Requirements Determination (Contd...)

- **Drawbacks**

- Tendency to avoid formal documentation
- Difficult to adapt to more general user audience
- Sharing data with other systems is often not considered
- Systems Development Life Cycle (SDLC) checks are often bypassed

Comparison of different requirement gathering technique

Characteristic	Interviews	Questionnaires	Observation	Document analysis	JAD	Prototyping
Information Richness	High	Medium to low	High	Low (passive) and old	High	Medium to High
Time Required	Can be extensive	Low to moderate	Can be extensive	Low to moderate	Dedicated period of time of all kinds of involved people	Moderate and can be extensive
Expense	Can be high	Moderate	Can be high	Low to moderate	High	High
Chance for Follow-up and probing	Good	Limited	Good	Limited	Good	Good
Confidentiality	Interviewee is known to interviewer	Respondent can be unknown	Observee is known to interviewer	Depends on nature of document	All the people know each other	Usually know each other
Involvement of Subject	Interviewee is involved and committed	Respondent is passive, no clear commitment	Interviewees may or may not be involved and committed depending on whether they know if they are being observed	None, no clear commitment	All kinds of people are involved and committed	Users are involved and committed
Potential Audience	Limited numbers, but complete responses from those interviewed	Can be quite large, but lack of response from some can bias results	Limited numbers and limited time of each	Potentially biased by which documents were kept or because document not created for this purpose	Potentially biased by the subordinator intentionally don't want to directly point out his superior's errors.	Limited numbers; it is difficult to diffuse or adapt to other potential users

3- Radical Methods for Determining System Requirements

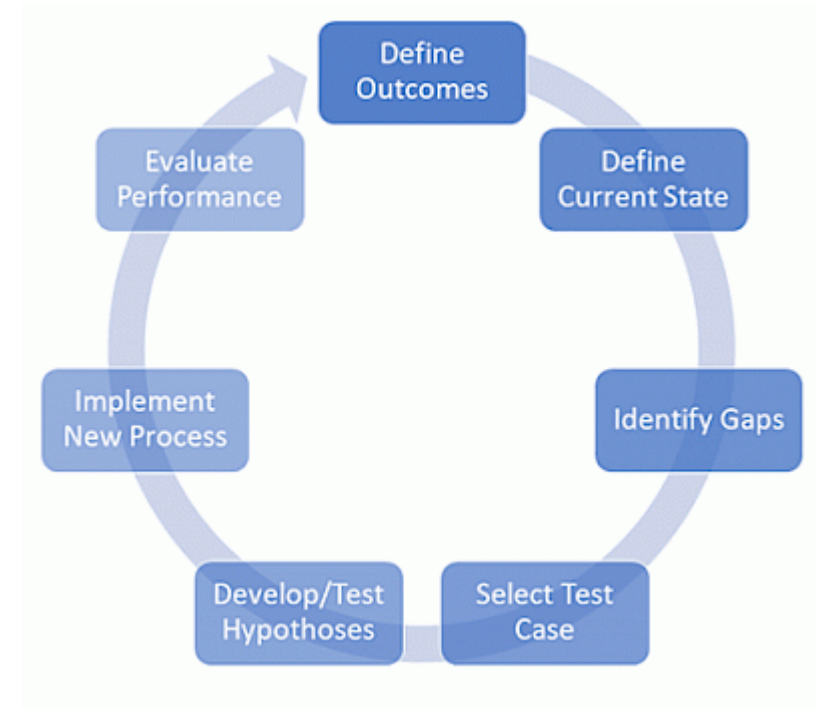
3.1 Business process re-engineering (BPR) is a business management strategy, originally pioneered in the early 1990s, focusing on the analysis and design of workflows and business processes within an organization.

- BPR aimed to help organizations fundamentally rethink how they do their work in order to improve customer service, cut operational costs, and become world-class competitors.
- Business process re-engineering (BPR) is the act of changing an organization's major functions with the goal of increasing efficiency, improving product quality, and/or decreasing costs. This starts with an in-depth analysis of the business' workflows and identifying key areas that need improvement. People who do this kind of work, often referred to as BPR specialists, are hired by companies to facilitate transitions to more standardized processes.

Business Process Reengineering

- **Business Process Reengineering Steps**

- Define outcomes
- Define current state
- Identify gaps
- Select test case
- Develop/test hypotheses
- Implement the new process
- Evaluate performance



3.2 Disruptive Technologies

- New technology can either be sustaining or disruptive. While sustaining technology depends on the incremental improvements in the already existing technology, disruptive technology is a completely new one. Hence, the practical application of such types of technology may not have been proven yet.
- Artificial intelligence, virtual reality, expert system, internet of thing, and e-commerce are some of the disruptive technologies significantly influencing the future.
- Disruptive technology is the technology that affects the normal operation of a market or an industry. It displaces a well-established product or technology, creating a new industry or market.

Structuring System Process Requirements

Process Modeling: Process modeling involves graphically representing the processes or actions that capture, manipulate, store and distribute data between the system and its environment and among components within the system. A common form of a process model is a data flow diagram (DFD)

Modeling of system process: Analysts use both process and data models to establish the specification of an information system. With a supporting tool, such as CASE tool, process and data models can also provide the basis for the automatic generation of an information system.

Deliverables and Outcomes from Process Modeling

In structured analysis, the primary deliverables from process modeling are set of coherent, interrelated data flow diagrams. Following are the deliverables for process modeling:

- ✓ Context DFD
- ✓ DFD of current physical system
- ✓ DFD of new logical system
- ✓ Through description of each DFD components

Data Flow Diagram (DFD)

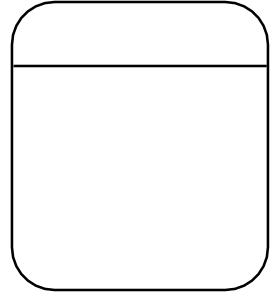
- A data flow diagram is a graphical representation of the "flow" of data through an information system, modeling its process aspects.
- A data flow diagram shows how data is processed within a system based on inputs and outputs. DFD has often been used for the following reasons:
 - ✓ Logical information flow of the system
 - ✓ Determination of physical system construction requirements.
 - ✓ Establishment of manual and automated systems requirements.

Data Flow Diagram (DFD): Symbols

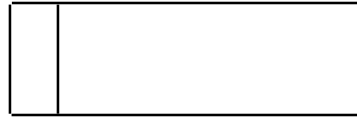
- There are two different sets of data flow diagram symbols, but each set consists of four symbols that represent the same things:
 - data flows,
 - data stores,
 - processes, and
 - sources/sinks (or external entities)

DFD Symbols and Definitions

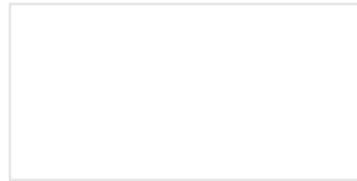
Process



Data store



**External
Entity**

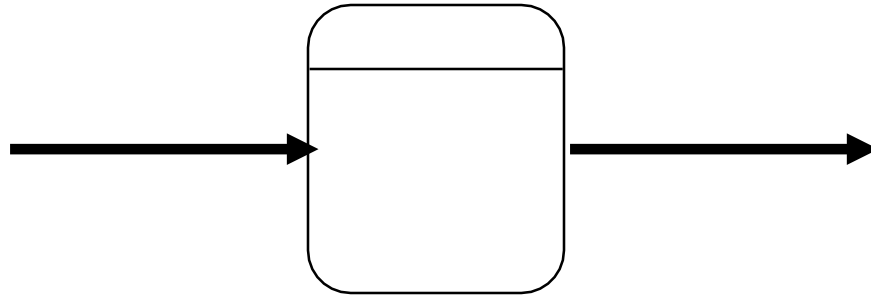


Data flow

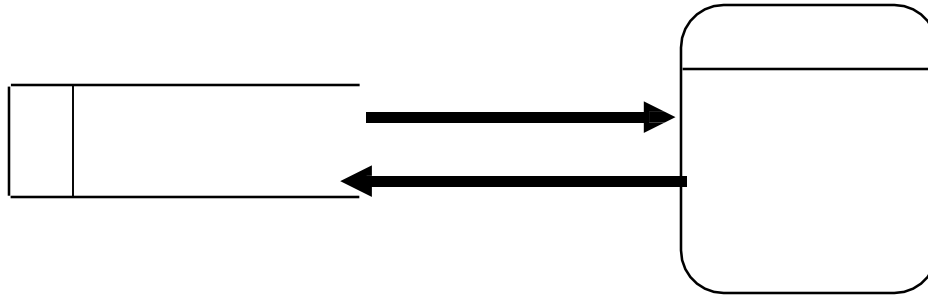


- **Process** - performs some action on data, such as creates, modifies, stores, delete, etc. Can be manual or supported by computer. The name consists of a verb followed by a singular noun. Example apply payment, verify order etc. A rounded rectangle represents a process. Processes are given IDs for easy referencing.
- **Data store** - information that is kept and accessed. May be in paper file folder or a database.
- **External entity** - is the origin or destination of data. Entities are external to the system. An external entity is a person, department, outside organization or other information systems. A rectangle represents external entities.
- **Data flow** - the flow of data into or out of a process, data store or entity. Straight lines with incoming and outgoing arrows are input and output data flow respectively.

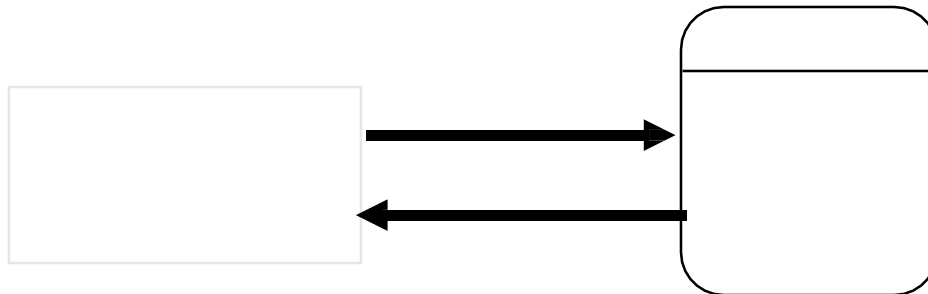
Rules for Drawing DFD's



A **minimum** of one data flow in and one data flow out of a process



A datastore **must** be connected to a process (either in, out, or both)

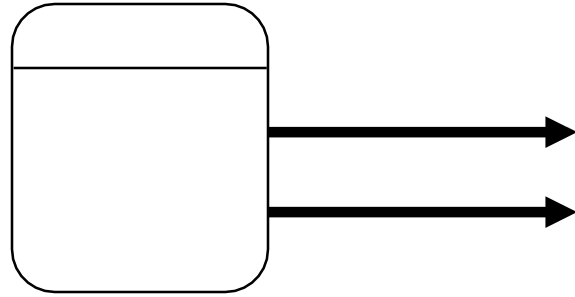


An external entity **must** be connected to a process (either in, out, or both)

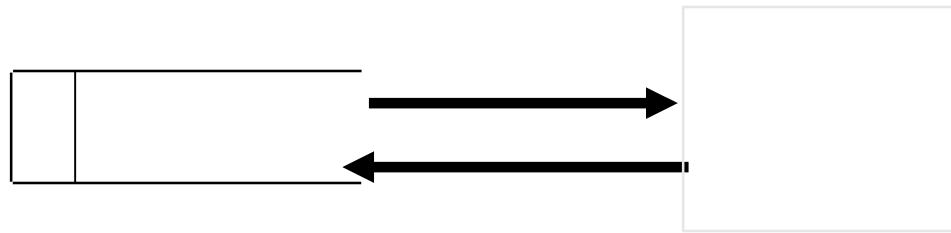


A single data flow **must** only flow one way

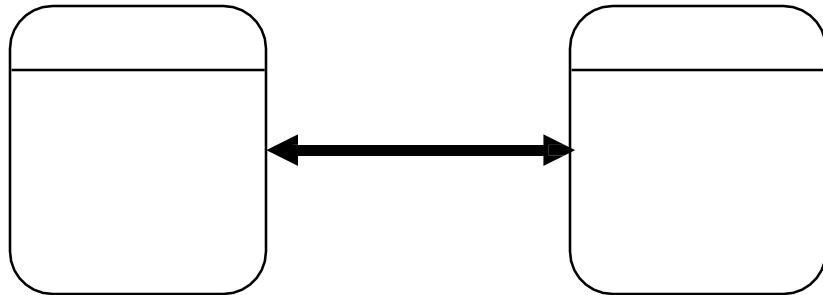
DFD: Common Mistakes



- Process has no data flowing into it, but has data flowing out.

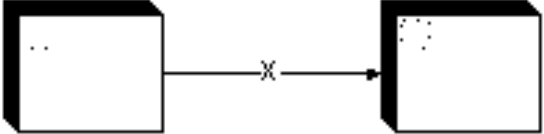
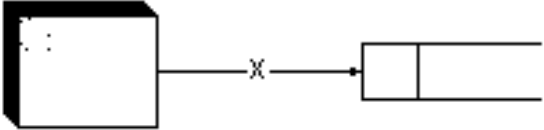
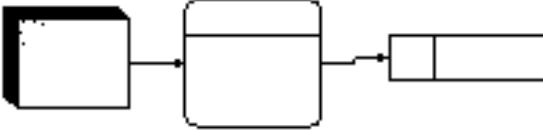
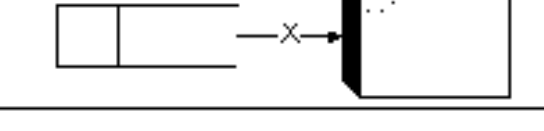
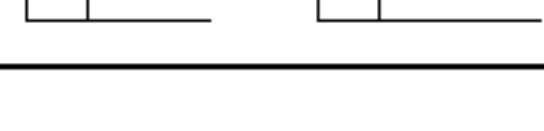


- Data store is hooked to external entity. This means external entity can read and write to your data file without auditing!!



- The data flow goes in two directions at once. Two or more arrows should be used to show the flow to and from each process.

Common DFD Mistakes

Wrong	Right	Description
		A source or a sink cannot provide data to another source or sink without some processing occurring.
		Data cannot move directly from a source to a data store without being processed.
		Data cannot move directly from a data store to a sink without being processed.
		Data cannot move directly from one data store to another without being processed.

Creating Data Flow Diagrams

Guidelines for developing data flow diagram:

1. Context Diagram-Level 0:
 - ✓ The context diagram must fit in one page.
 - ✓ Process name in context diagram should be the name of the information system. For example: Registration system.
 - ✓ The context level diagram gets the number 0.
2. Unique name for levels:
 - ✓ Unique names should be used for each set of symbols.
 - ✓ For example, there can be only one entity in all levels of the data flow diagrams.

Creating Data Flow Diagrams

3. No Cross Line in DFD:

- ✓ One way to achieve this is to restrict number of process in DFD.
- ✓ Another way is to duplicate an external entity or data store using a special notation such as asterisk.

4. Numbering Convention:

- ✓ Use a unique reference number for each process symbol.
- ✓ Other process number are in the hierarchy of:
 - ✓ (1,2,3.....)
 - ✓ (1.1,1.2,.....2.1,2.2,.....)

Level of DFD

1. **Level 0 (Context Diagram):** This is the highest-level DFD and provides an overview of the entire system. It shows the boundaries of the system and the interactions between the system and external entities, but it doesn't go into detail about the internal processes.
2. **Level 1 DFDs:** These diagrams expand on the context diagram and provide more detail about the processes, data flows, and data stores within the system. Each process in a Level 1 DFD can be further decomposed into lower-level processes in subsequent DFDs.
3. **Level 2 DFDs:** If necessary, Level 1 DFDs can be further decomposed into Level 2 DFDs. Level 2 DFDs provide even more detail about the processes and data flows within a system, breaking down the processes from Level 1 into their sub-processes.
4. **Subsequent Levels:** Depending on the complexity of the system, you can continue to create lower-level DFDs (e.g., Level 3, Level 4, etc.) to provide increasingly detailed views of the system's processes and data flows. These lower-level DFDs help in understanding the system's operations at a granular level.

Context Level Diagrams

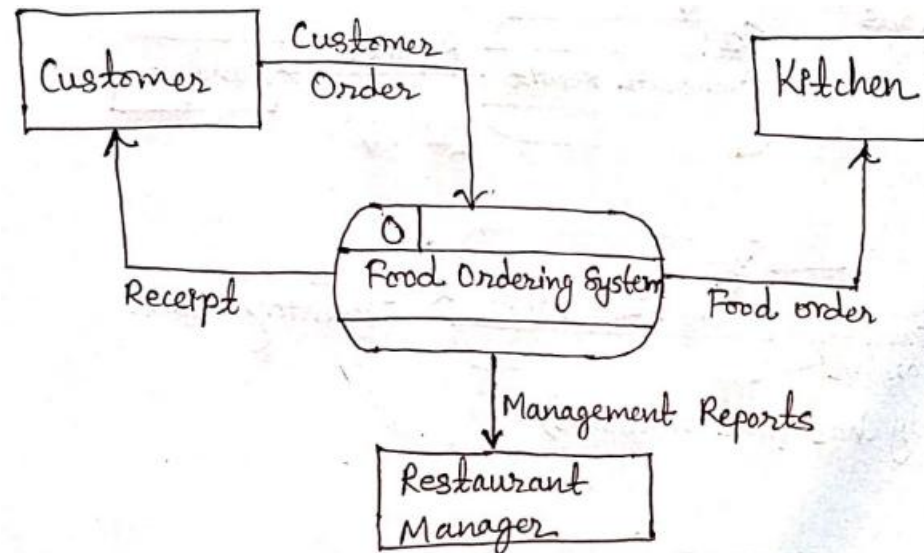
A context diagram gives an overview and it is the highest level in a data flow diagram containing only one process representing the entire system.

- All external entities are shown on the context diagram as well as major data flow to and from them.
- The diagram does not contain any data storage.
- The single process in the context level diagram, representing the entire system, can be exploded to include the major process of the system in the next level diagram , which is termed as diagram 0.

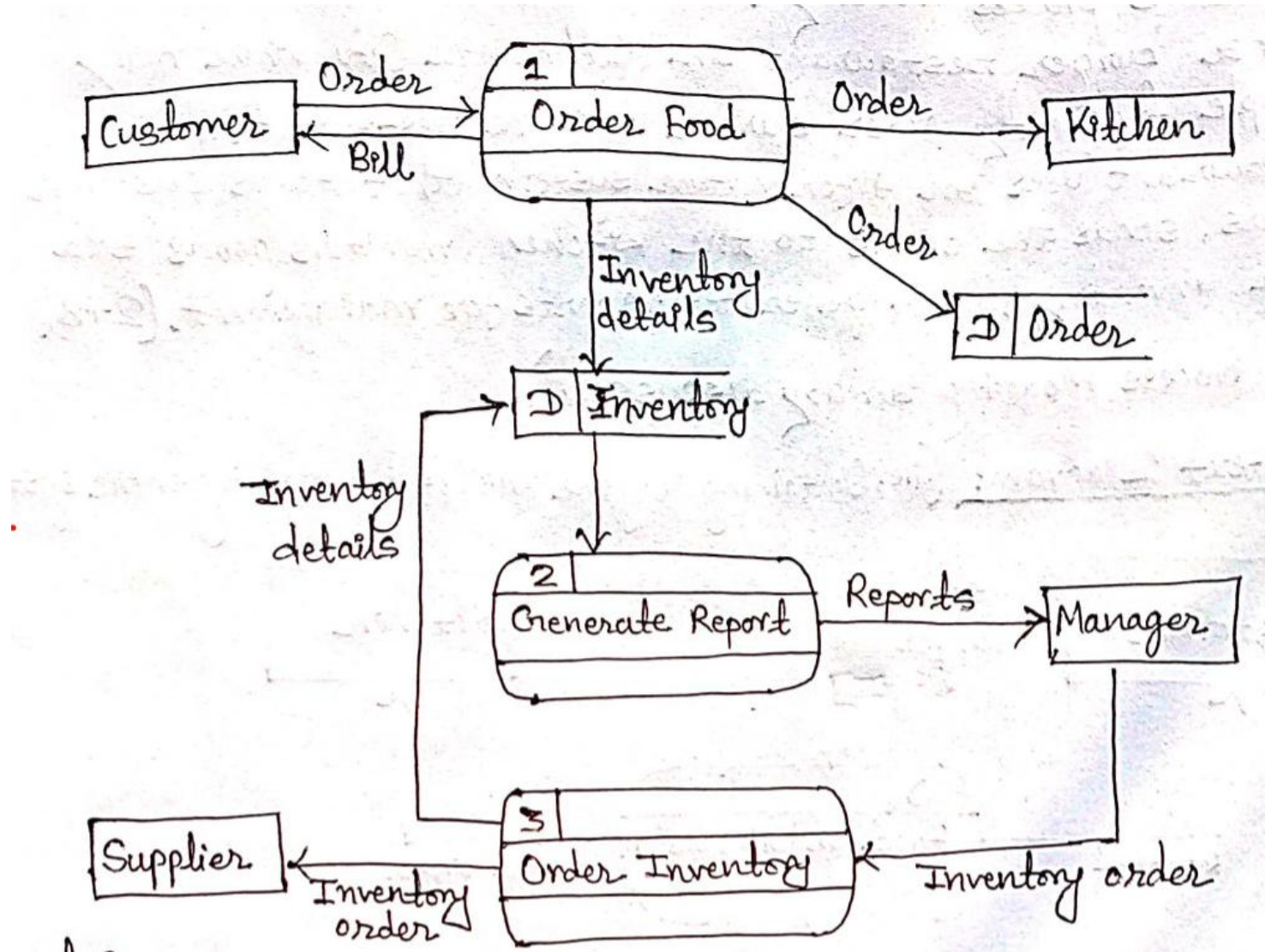
Important Questions

What is process modeling? Draw context diagram and DFD for a burger restaurant in Kathmandu City where many people frequently order burger at the restaurant. The restaurant uses an information system that takes customer orders, sends the orders to the kitchen, monitors goods sold and inventory, and generates reports for management.

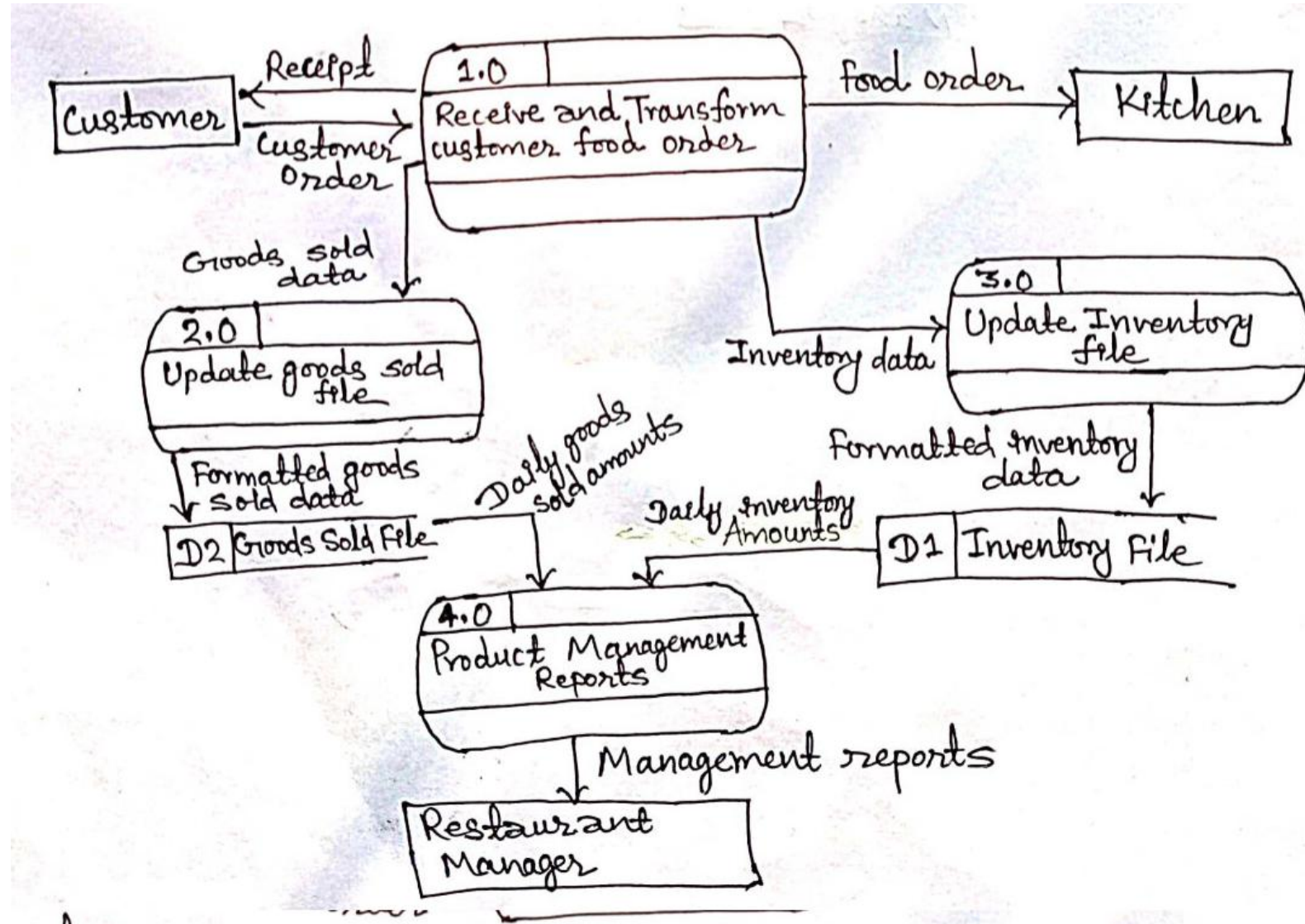
Ans----Context Diagram (i.e. Containing only one process representing entire system.



Level 1 DFD



Level 2 DFD



Logical DFD Vs Physical DFD

- A logical data flow diagram focuses on the business and how the business operates. It is not concerned with how the system will be constructed. We can ignore implementation specifics such as computer configuration, data storage, technology, etc.
- A Physical data flow diagram shows how the system will be implemented including the hardware, software, files, processes described in the logical data flow diagrams are implemented correctly to achieve the goal of the business.
- Context diagrams (Level 0), Level 1, and Level 2 Data Flow Diagrams (DFDs) are inherently **logical DFDs**, focusing on *what* the system does (business processes, data flow) rather than *how* it's implemented (hardware, software).

Logical DFD Vs Physical DFD

The benefit of logical data flow diagram:

- It is based on the business events and independent of a particular technology.
- It allows analysts to understand the business being started.
- Systems implemented based on logical DFD will be easier to maintain.
- Physical DFD can be easily formed by modifying a logical DFD

The benefit of Physical data flow diagram:

- Describe processes in more detail than logical DFDs
- Identifies temporary data storage.
- Clarifies which processes are manual and which are automated
- Adds control to ensure the processes are done properly.

Logic Modeling

- A logic model is a graphic representation that presents the shared relationships among program's resources, activities, outputs, outcomes, and impact.
- Data flow diagrams do not show the logic inside the processes –what occurs within a process? How input data is converted into output information. Logic modeling involves representing the internal structure and functionality of processes depicted on a DFD.
- The logic model has four components:
 1. **Needs-** Needs are about the problem and why its important to address it. What issues are we trying to address?
 2. **Inputs-** These are the things that contribute to addressing the need. What resources are we investing?
 3. **Activities-** These describe the things that the inputs allow to happen. What are we doing with these resources?
 4. **Outcomes-** Outcomes are usually expressed in terms of measures of success. What difference are we hoping to make?

Modeling Logic with Decision Tables:

Decision tables are a concise visual representation for specifying which actions to perform depending on given conditions. They are algorithms whose output is a set of actions. A decision table is an excellent tool to use in both testing and requirements management. Decision tables are used to model complicated logic. A decision table makes the same requirements clearer to understand: In a decision table, conditions are usually expressed as true(T) or false (F). Each column in the table corresponds to a rule in the business logic that describes the unique combination of circumstances that will result in the actions.

Let's take an example scenario for an ATM where a decision table would be of use.

A customer requests a cash withdrawal. One of the business rules for the ATM is that the ATM machine pays out the amount if the customer has sufficient funds in their account or if the customer has the credit granted. Already, this simple example of a business rule is quite complicated to describe in the text.

Steps to create decision tables:

Step 1: Analyze the requirement and create the first column

Requirements: “ Withdrawal is granted if the requested amount is covered by the balance or if the customer is granted credit to cover the withdrawal amount”. Express conditions and resulting actions in a list so that they are either TRUE or FALSE. In this case, there are two conditions, “withdrawal amount $\leq$ balance” and “credit granted”. There is one result, the withdrawal is granted.

Conditions
Withdrawal amount $\leq$ balance
Credit granted
Actions
Withdrawal granted

Steps to create decision tables:

Step 2: Add column

Calculate how many columns are needed in the table. The number of columns depends on the number of conditions and the number of alternatives for each condition. If there are two conditions and each condition can be either true or false, you need 4 columns. If there are three conditions, there will be 8 columns, and so on. Mathematically, the number of columns is $2^{\text{conditions}}$. In this case $2^2 = 4$ columns.

Number of Conditions	Number of Columns	Conditions				
1	2	Withdrawal amount<=balance	T	F	T	F
2	4	Credit granted	T	T	F	F
3	8	Actions				
4	16					
5	32	Withdrawal granted				

Steps to create decision tables:

Step 3: Reduce the table

Mark insignificant values with “-”. If the requested amount is less than or equal to the account balance it does not matter if credit is granted. In the next step, you can delete the columns that have become identical.

Conditions				
Withdrawal amount<=balance	T	F	T	F
Credit granted	-	T	-	F
Actions				
Withdrawal granted				

Check for invalid combinations. Invalid combinations are those that cannot happen, for example, when someone is both an infant and a senior. Mark them somehow, e.g. with an “X”. In this example, there are no invalid combinations.

Steps to create decision tables:

Step 4: Determine actions

Enter actions for each column in the table. You will be able to find this information in the requirements. Name the column (the rules). They may be named R1/Rule 1, R2/Rule 2, and so on, but you can also give them more descriptive names.

Conditions			
Withdrawal amount<=balance	T	F	F
Credit granted	-	T	F
Actions			
Withdrawal granted	T	T	F

Steps to create decision tables:

Step 5: Write test cases

Write test cases based on the table. At least one test case per column gives full coverage of all business rules.

- **Test case for R1:** balance =200, requested withdrawal =200. Expected result: withdrawal granted.
- **Test case for R2:** balance =100, requested withdrawal =200, credit granted. Expected result: Withdrawal granted
- **Test case for R3:** balance =100, requested withdrawal =200, no credit. Expected result: Withdrawal denied.

Decision tables:

Advantage:

- They are easier to draw.
- The decision table can be easily changed according to the situation.
- A small table can replace several pages of a flow chart.
- The requirements become much clearer.
- Decision tables make it possible to detect combinations of conditions.

Disadvantage:

- When there are too many alternatives decision table cannot list them all.
- It cannot express a complete sequence of operations to solve a problem.
- They only present partial solutions.

Modeling logic with decision tree:

- A decision tree is a graphical model describing decisions and their possible outcomes. It defines all actions that result from various combinations of conditions as per logical rules. The graphical representation of conditions and outcomes resembles the branches of the tree. The branches of a tree depend on the logical alternatives.
- It is drawn horizontally with the root on the left side and branches to the right. Though it is easy to construct, read, and update a decision tree for a simple problem as the complexity increases it becomes tedious to draw and update.
- There are three types of decision tree nodes:
 - 1. Decision Node**
 - 2. Chance Node**
 - 3. Terminal Node**

Modeling logic with decision tree:

1. **Decision Node:** Often represented by squares showing decisions that can be made. Lines starting from a square show all distinct options available at a node.
2. **Chance Node:** Often represented by circles showing chance outcomes. Chance outcomes are events that can occur but are outside the ability of the decision-maker to control.
3. **Terminal Node:** Often represented by triangles or by lines having no further decision nodes or chance nodes. Terminal nodes depict the final outcomes of the decision-making process.

Hungry
or not?

Yes

Is it
midday?

Yes

eat lunch

Yes

eat dinner

No

Is it after
6pm?

No

have a
snack

No

Work
to do?

Yes

do work

Yes

watch tv

No

Have you
worked
out?

No

70% chance of rain

go to
the gym

30% chance of rain

go running

Modeling logic with decision tree:

Advantage:

- ✓ It expresses the logic of if then else in pictorial form.
- ✓ It helps the analyst to identify the actual decision to be made.
- ✓ It is used to verify logic and problems that involve a few complex decisions and a limited number of actions.

Disadvantage:

- ✓ There is the absence of information in its format to take other combinations of conditions to test.
- ✓ A large number of branches with many paths will confuse rather than help in analysis.

Modeling logic with Pseudocodes:

- Pseudocode is an informal way of programming description that does not require any strict programming language syntax.
- It is used for creating an outline or a rough draft of a program. It has no syntax like any of the programming languages and thus can't be compiled by the computer.

Example:

```
if "1"  
  print response  
  "I am case 1"
```

```
if "2"  
  print response  
  "I am case 2"
```

Modeling logic with Pseudocodes:

Advantage:

- Improves the readability of any approach.
- It is one of the best approaches to start the implementation of an algorithm.
- Acts as a bridge between the program and the algorithm of flowchart.
- Explains what exactly each line of a program should do.

Disadvantage:

- No standard
- Time consuming
- Hard to verify
- Difficult to map processes.

Guidelines for Selecting Appropriate Tools

- Use the following guidelines for selecting the most appropriate tool that would suit your requirements –
 - Use DFD at high or low level analysis for providing good data flow & system documentations.
 - Use data dictionary to simplify the structure for meeting the data requirement of the system.
 - Use decision tables when there are a large number of conditions to check and logic is complex.
 - Use decision trees when sequencing of conditions is important and if there are few conditions to be tested.
 - Use structured English if there are many loops and actions are complex.

Structuring System Data Requirements

Structuring systems and data requirements concentrates on the definition, structure, and relationships within data. The most common format used for data modeling is entity relationship diagramming.

❑ **Conceptual data modeling:**

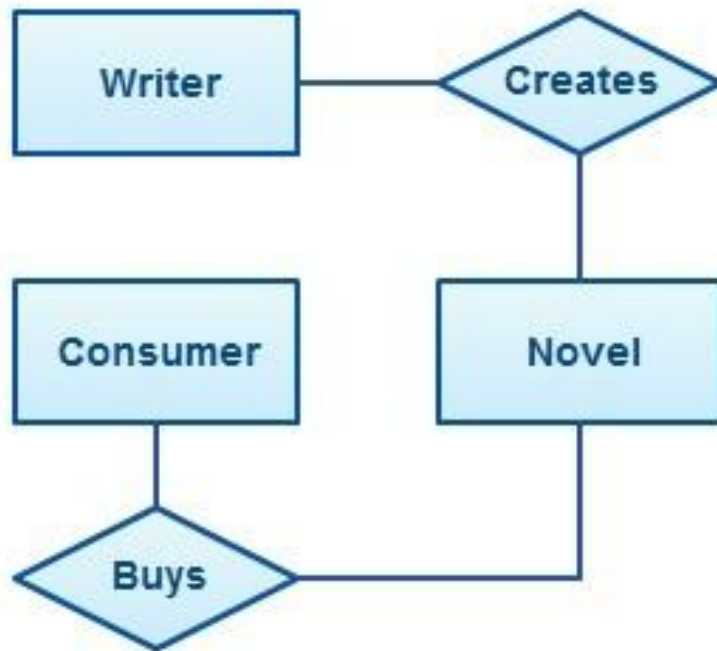
A conceptual data model is a map of concepts and their relationships used for databases. It is a representation of organizational data. Conceptual data modeling is typically done in parallel with other requirements analysis and structuring steps during system analysis. On larger systems, a subset of the project team concentrates on data modeling while other team members focus attention on process or logic modeling. Analysts develop a conceptual data model for the current system.

❑ **Deliverables and Outcomes:**

The primary deliverable from the conceptual data modeling is the entity relationship diagram. Another deliverable from conceptual data modeling is a full set of entries about data objects to be stored in the project repository.

Entity Relationship Model

- Entity-Relationship (ER) modeling is a graphical approach to database design.
- It is a high-level data model that defines data elements and their relationship for a specified software system.
- An ER model is used to represent real-world objects.



The elements inside **rectangles** are called **entities** while the items inside **diamonds** denote the **relationships** between entities.

The Entity-Relationship Model

- An entity relationship model, also called an entity- relationship (ER) **diagram**, is a graphical representation of entities and their relationships to each other, typically used in computing in regard to the organization of data within databases or information systems.
- An entity relationship diagram (ERD) shows the relationships of entity sets stored in a database.
- An entity in this context is a component of data.
- In other words, ER diagrams illustrate the logical structure of databases

The E-R Model Contd..

- A systems analyst uses entity relationship data model (E-R model) as a conceptual data model.
- A conceptual data model is a detailed model that captures the overall structure of organizational data while being independent of any database management system or other implementation consideration.
- And an E-R model is a detailed, logical representation of the data for an organization or for a business area.

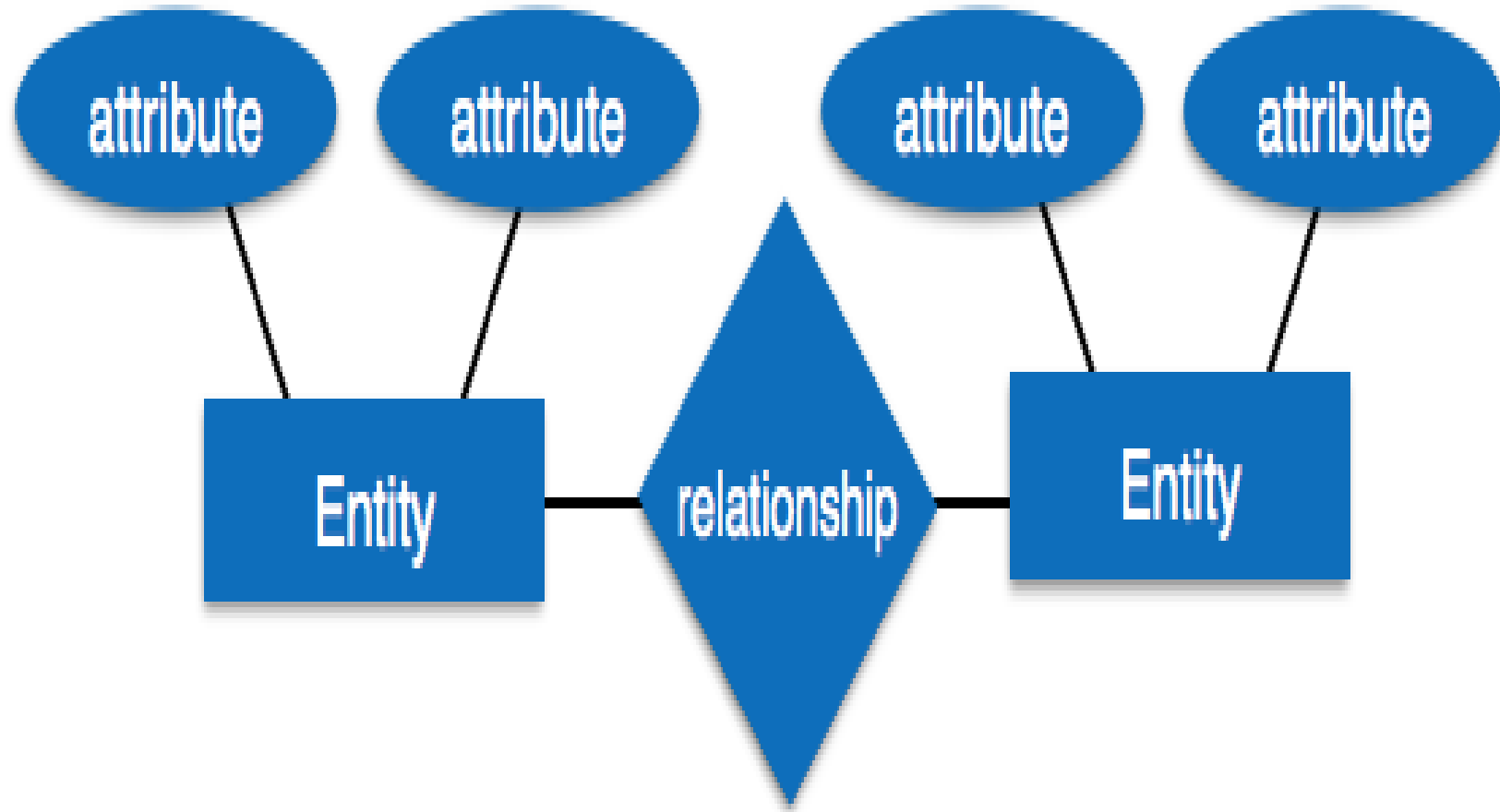
The Entity-Relationship Model Cont..

- The E-R model is expressed in terms of entities in the business environment, the relationships or associations among those entities, and the attributes or properties of both the entities and their relationships.
- An E-R model is normally expressed as an entity relationship diagram (E-R diagram), which is a graphical representation of an E-R model.
- It has three basic concepts: **entities, attributes, and relationships.**
- Entity-Relationship model is a type of database model based on the notion of real world entities and relationship among them. We can map real world scenario onto ER database model.
- ER Model creates a set of entities with their attributes, a set of constraints and relation among them.
- ER Model is best used for the conceptual design of database.

The E-R Model Contd..

- Entity-Relationship model is a type of database model based on the notion of real world entities and relationship among them. We can map real world scenario onto ER database model.
- ER Model creates a set of entities with their attributes, a set of constraints and relation among them.
- ER Model is best used for the conceptual design of database.
- ER Model can be represented as follows :

The E-R Model Cont...



Entity

- *An entity can be a real-world object*, either animate or inanimate, that can be *easily identifiable*.
 - For example, in a school database, **students, teachers, classes, and courses offered** can be considered as **entities**.
 - All these entities have some *attributes or properties* that give them their identity.
- An **entity set** is a collection of similar types of entities.
 - An entity set may contain *entities with attribute sharing similar values*.
 - For example,
 - a Students set may contain all the students of a school;
 - likewise a Teachers set may contain all the teachers of a school from all faculties. Entity sets need not be disjoint.

Entity

- **Examples of entities:**
- **Person:** Employee, Student, Patient
- **Place:** Store, Building
- **Object:** Machine, product, and Car
- **Event:** Sale, Registration, Renewal
- **Concept:** Account, Course



Entity Sets *customer* and *loan*

customer-id customer- customer- customer- loan- amount
 name street city number

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield

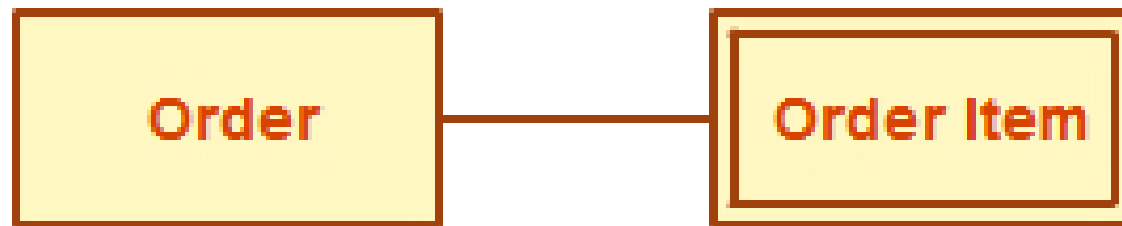
customer

L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300

loan

Weak Entity

- A weak entity is an entity that **depends on the existence of another entity**.
- In more technical terms it can be defined as an entity that cannot be identified by its own attributes.
- It uses a foreign key combined with its attributes to form the primary key.
- An entity like **order item** is a good example for this.
 - The order item will be meaningless without an order so it depends on the existence of order.



WEAK ENTITY SETS

An entity set that does not have a primary key is referred to as a **weak entity set**

- We underline the discriminator of a weak entity set with a dashed line.
- We put the identifying relationship of a weak entity in a double diamond.
- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



Strong Entity Set	Weak Entity Set
Strong entity set always has a primary key.	It does not have enough attributes to build a primary key.
It is represented by a rectangle symbol.	It is represented by a double rectangle symbol.
It contains a Primary key represented by the underline symbol.	It contains a Partial Key which is represented by a dashed underline symbol.
The member of a strong entity set is called as dominant entity set.	The member of a weak entity set called as a subordinate entity set.
Primary Key is one of its attributes which helps to identify its member.	In a weak entity set, it is a combination of primary key and partial key of the strong entity set.
In the ER diagram the relationship between two strong entity set shown by using a diamond symbol.	The relationship between one strong and a weak entity set shown by using the double diamond symbol.
The connecting line of the strong entity set with the relationship is single.	The line connecting the weak entity set for identifying relationship is double.

Attributes

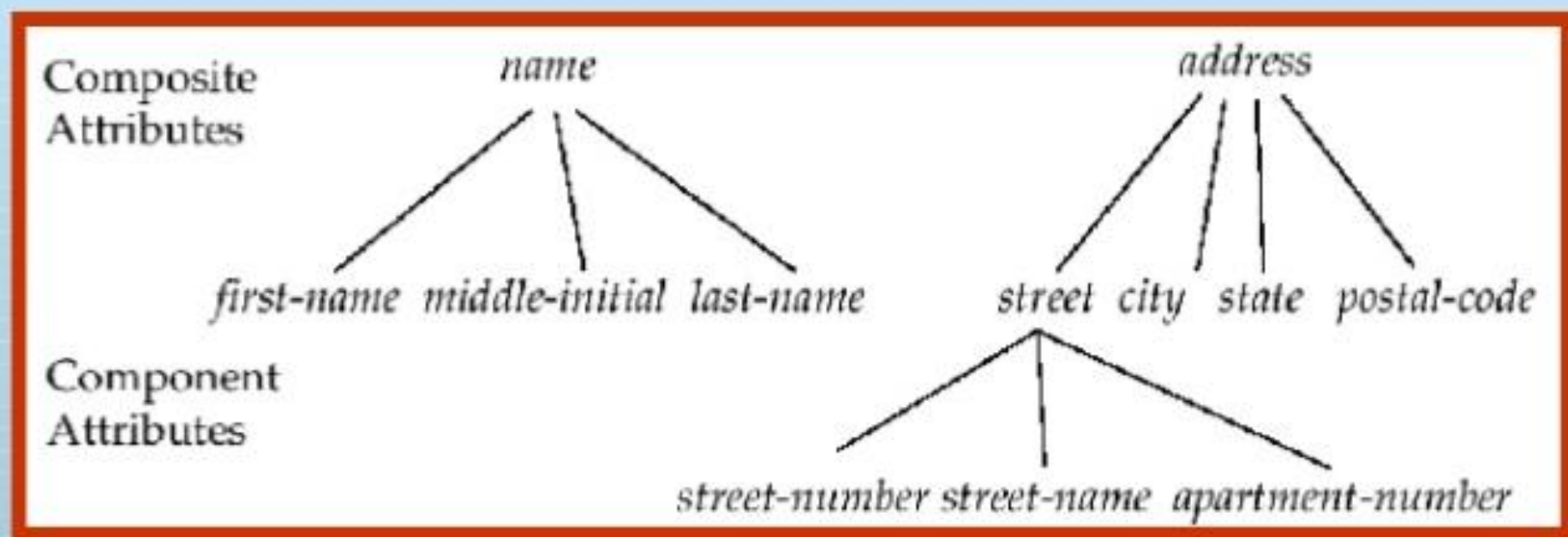
- Entities are represented by means of their *properties*, called attributes.
- All attributes have values.
- For example, a **student entity** may have **name**, **class**, and **age** as attributes.
- *There exists a domain or range of values that can be assigned to attributes.*
- For example, a **student's name** cannot be a numeric value. It has to be *alphabetic*. A **student's age cannot be negative**, etc.

Types of Attributes

- **Simple attribute** – Simple attributes are atomic values, which cannot be divided further. For example, **a student's phone number is an atomic value of 10 digits.**
- **Composite attribute** – Composite attributes are made of more than one simple attribute. For example, **a student's complete name may have first_name and last_name.**

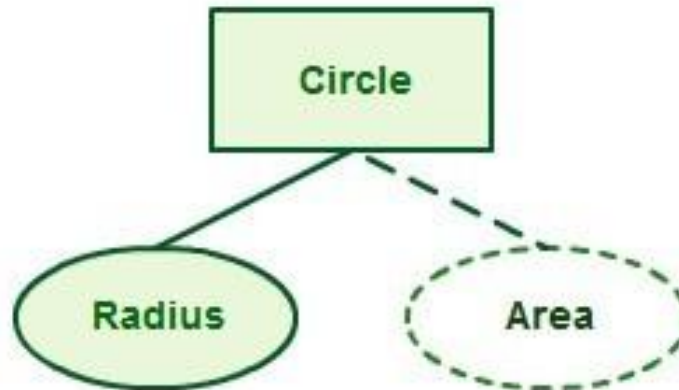


Composite Attributes

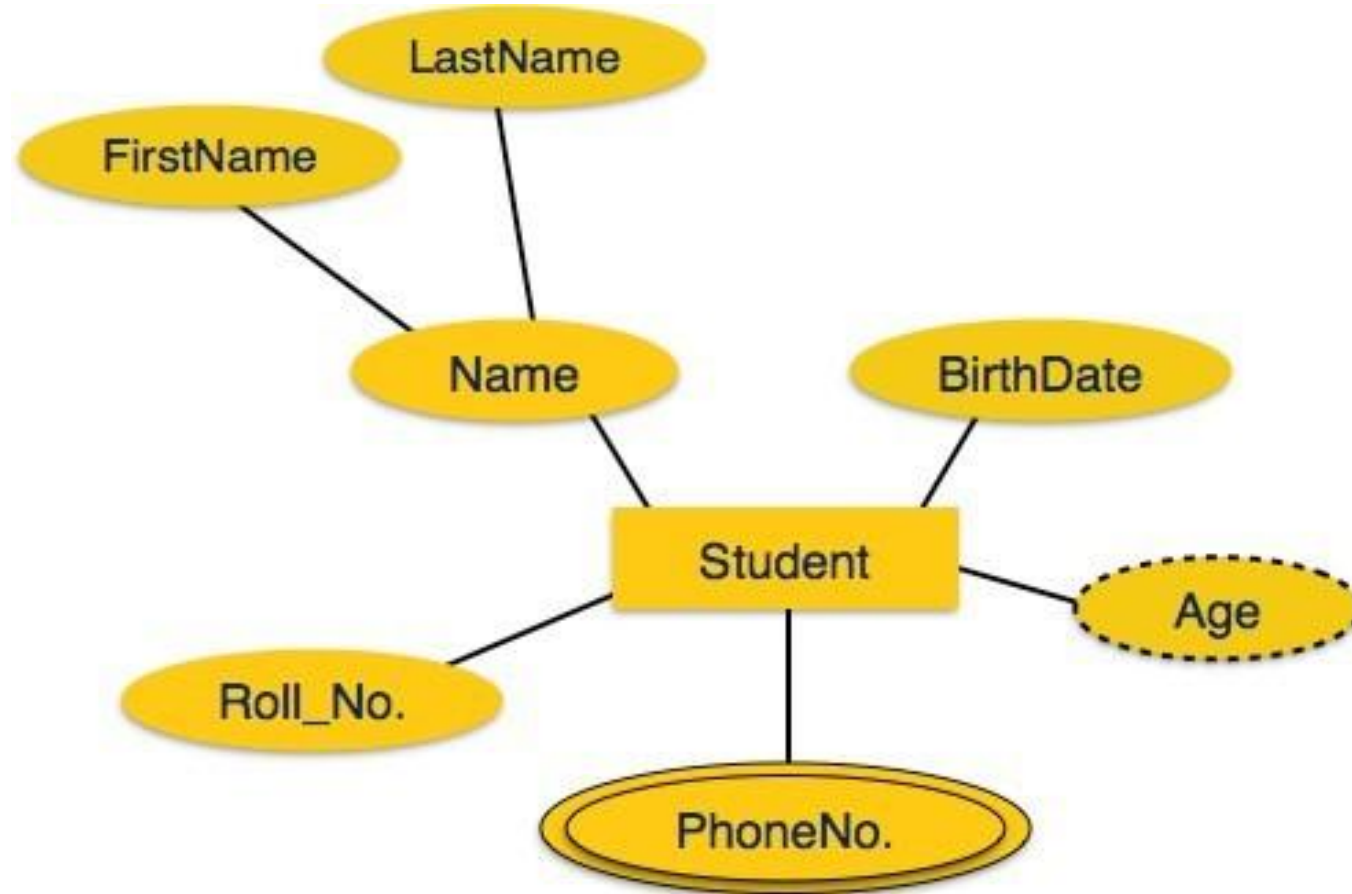


Types of Attributes (contd.,)

- **Derived attribute** – Derived attributes are the attributes that do not exist in the physical database, but their **values are derived from other attributes** present in the database.
- For example, average_salary in a department should not be saved directly in the database, instead it can be derived.
- For another example, age can be derived from data_of_birth.
- For example for a circle the area can be derived from the radius.



- **Derived** attributes are depicted by dashed ellipse.

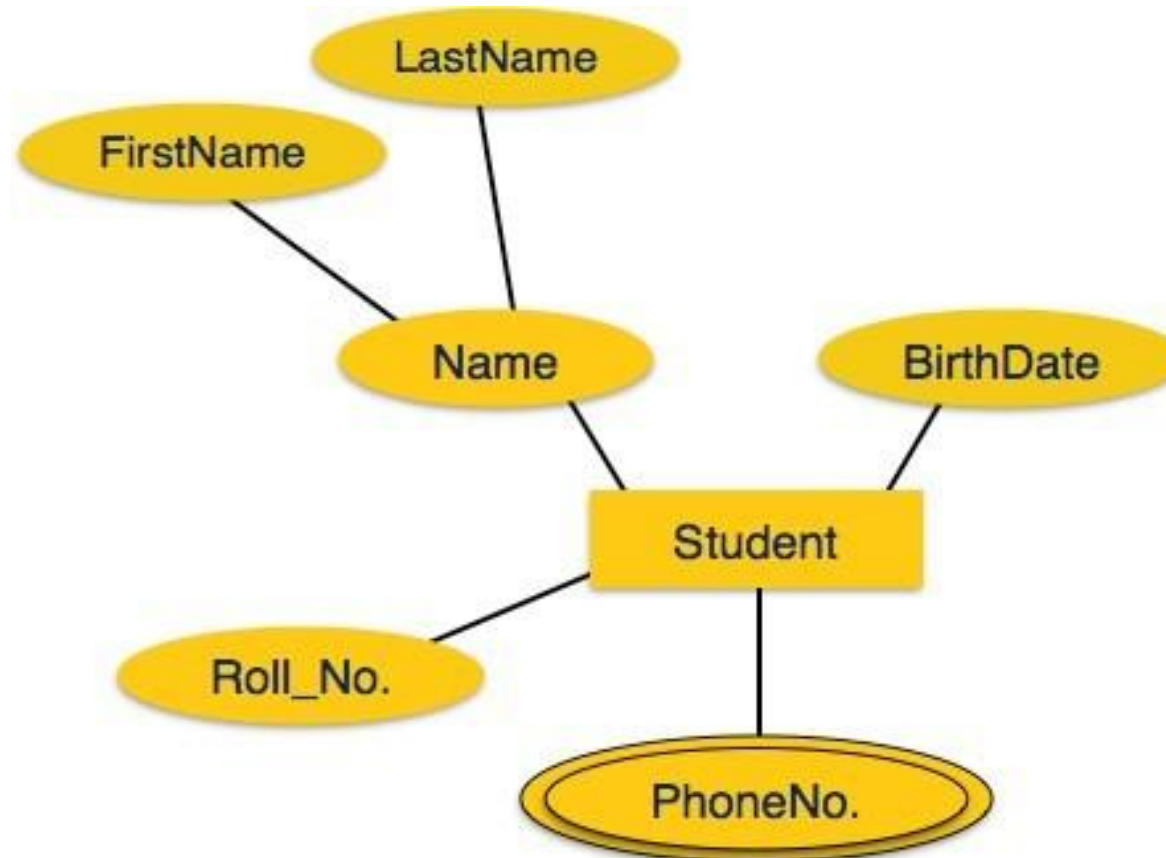


Types of Attributes (contd.,)

- **Single-value attribute** – Single-value attributes **contain single value**. For example – Social_Security_Number.
- **Multi-value attribute** – Multi-value attributes may **contain more than one values**. For example, a person can have more than one phone number, email_address, etc.
- For example a teacher entity can have multiple subject values.



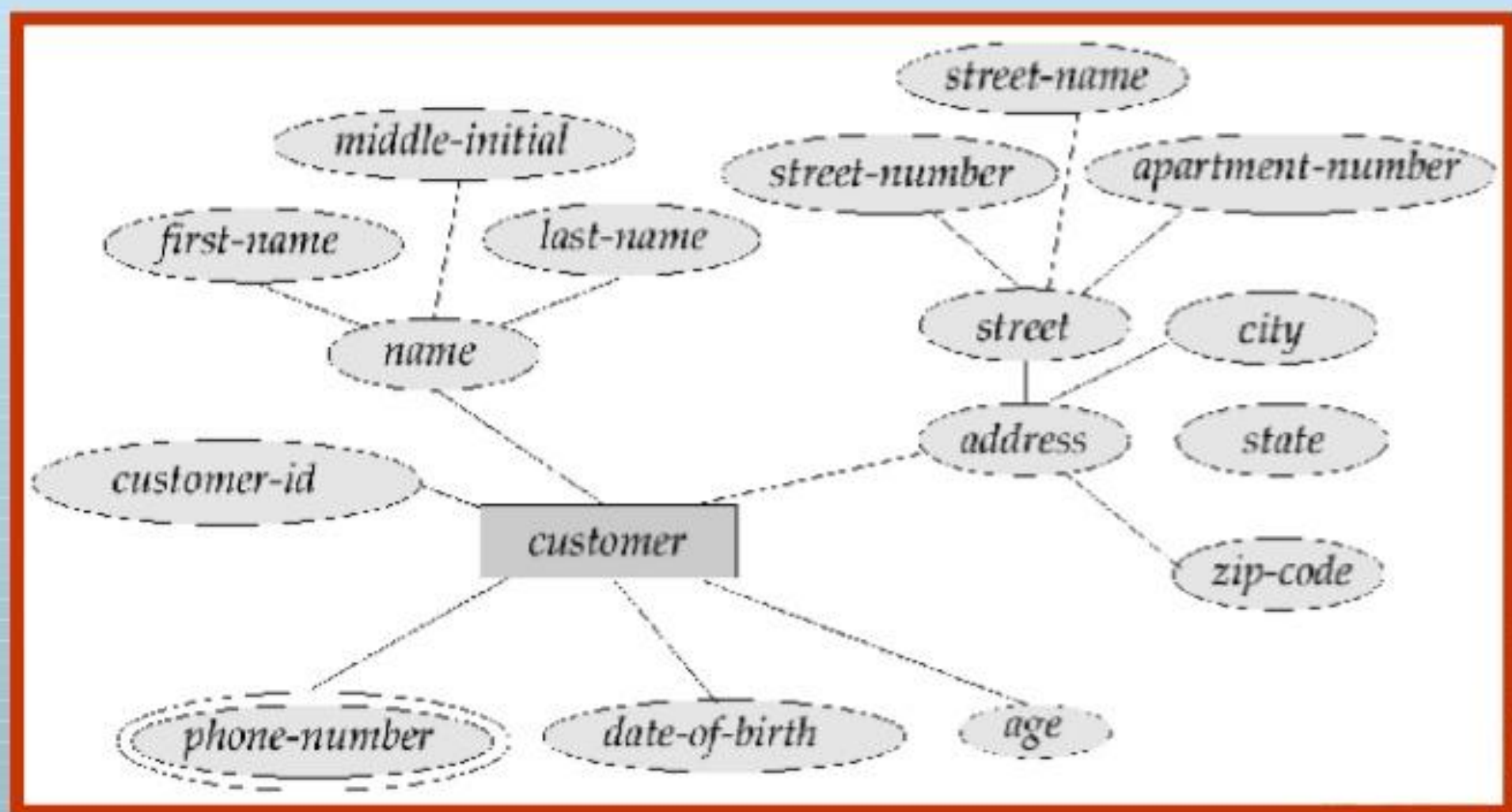
- **Multivalued** attributes are depicted by double ellipse.



Types of Attributes	Description
Simple attribute	Simple attributes can't be divided any further. For example, a student's contact number. It is also called an atomic value.
Composite attribute	It is possible to break down composite attribute. For example, a student's full name may be further divided into first name, second name, and last name.
Derived attribute	This type of attribute does not include in the physical database. However, their values are derived from other attributes present in the database. For example, age should not be stored directly. Instead, it should be derived from the DOB of that employee.
Multivalued attribute	Multivalued attributes can have more than one values. For example, a student can have more than one mobile number, email address, etc.



E-R Diagram With Composite, Multivalued, and Derived Attributes



Entity-Set and Keys

- **Key** is an attribute or collection of attributes that uniquely identifies an entity among entity set.
- For example, the **roll_number** of a student makes him/her identifiable among students.
- **Super Key** – A set of attributes (one or more) that collectively identifies an entity in an entity set.
- **Candidate Key** – A minimal super key is called a candidate key.
An entity set may have more than one candidate key.
- **Primary Key** – A primary key is one of the candidate keys chosen by the database designer to uniquely identify the entity set.

PRIMARY KEY

- A primary key is a candidate key that is most appropriate to be the main reference key for the table. As its name suggests, it is the primary key of reference for the table and is used throughout the database to help establish relationships with other tables.
- **The primary key must contain unique values, must never be null and uniquely identify each record in the table**

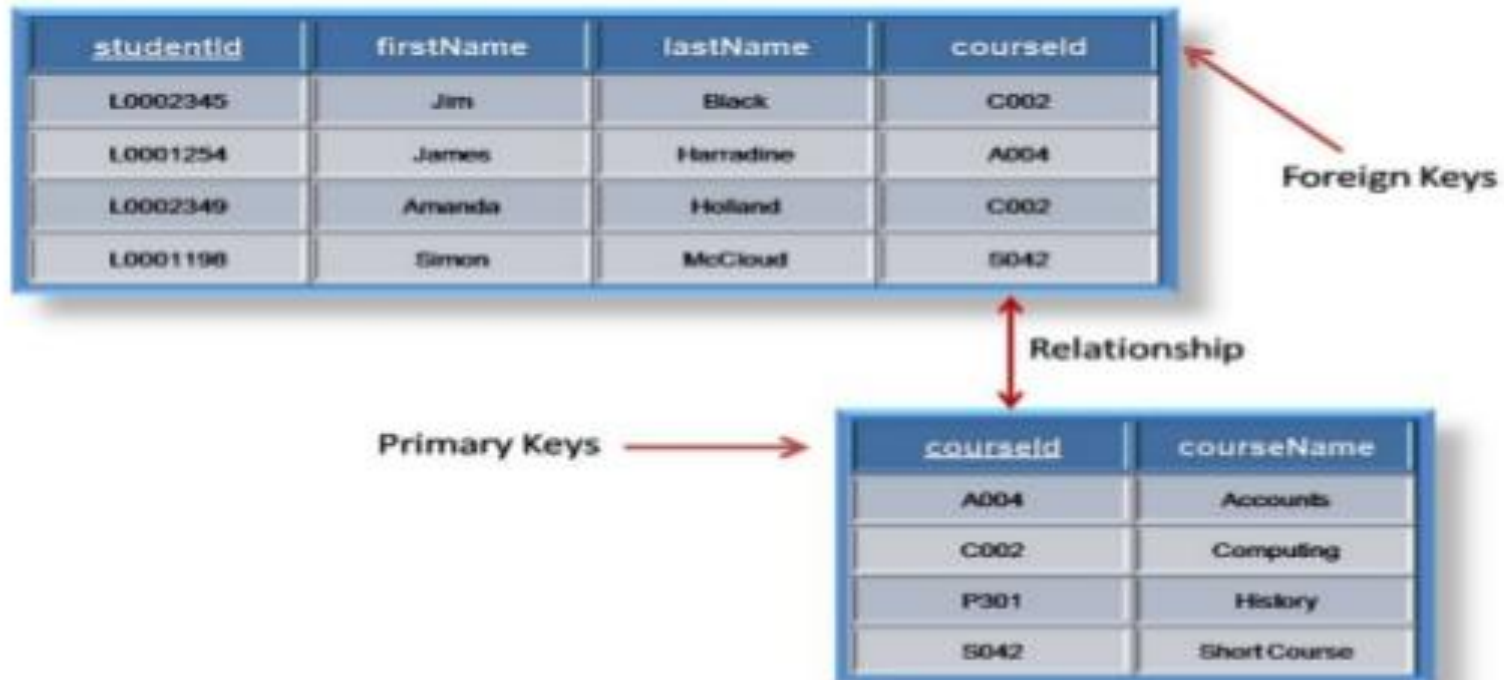
Primary Keys



<u>StudentId</u>	firstName	lastName	courseId
L0002345	Jim	Black	C002
L0001254	James	Harradine	A004
L0002349	Amanda	Holland	C002
L0001198	Simon	McCloud	S042
L0023487	Peter	Murray	P301
L0018453	Anne	Norris	S042

FOREIGN KEY

- A foreign key is generally a primary key from one table that appears as a field in another where the first table has a relationship to the second. In other words, if we had a table A with a primary key X that linked to a table B where X was a field in B, then X would be a foreign key in B



Relationship

- The association among entities is called a relationship.
- For example, an employee **works at** a department, a student **enrolls** in a course.
- Here, Works_at and Enrolls are called relationships.
- For example, the entity “**carpenter**” may be related to the **entity** “**table**” by the **relationship** “**builds**” or “**makes**”. Relationships are represented by diamond shapes and are labeled using verbs.



Degree of Relationship

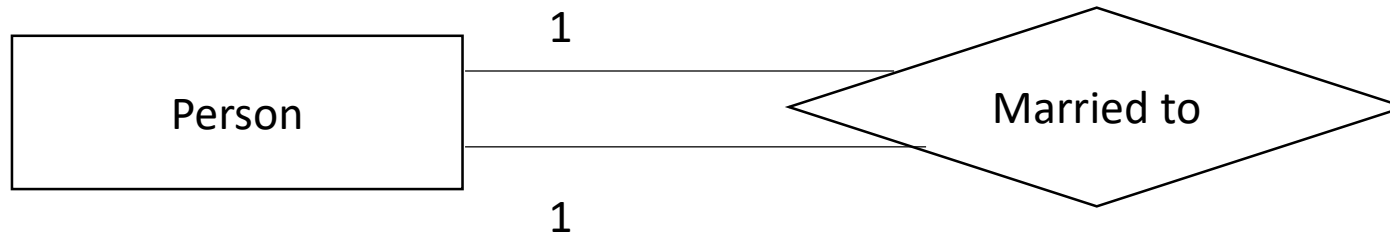
The number of entity sets that participate in the relationship set is called the degree of the relationship set. On the basis of degree, relationships can be divided as below:

- Unary Relationship
- Binary Relationship
- N-ary Relationship

Unary Relationship

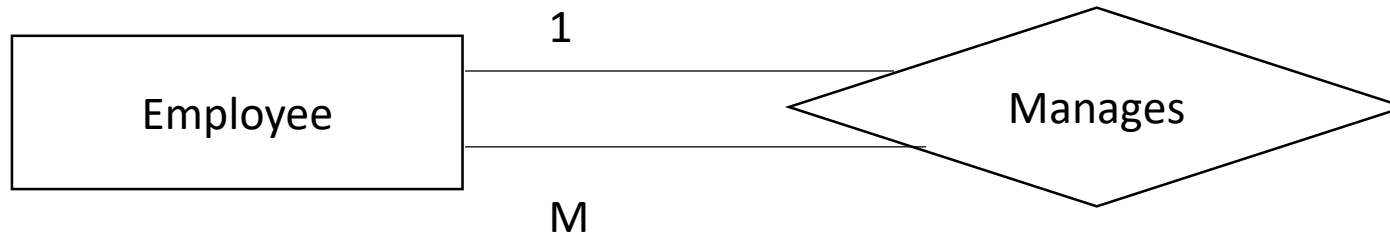
Only one entity is set to participate in a relationship.

- **One to one (1:1) unary relationship:**



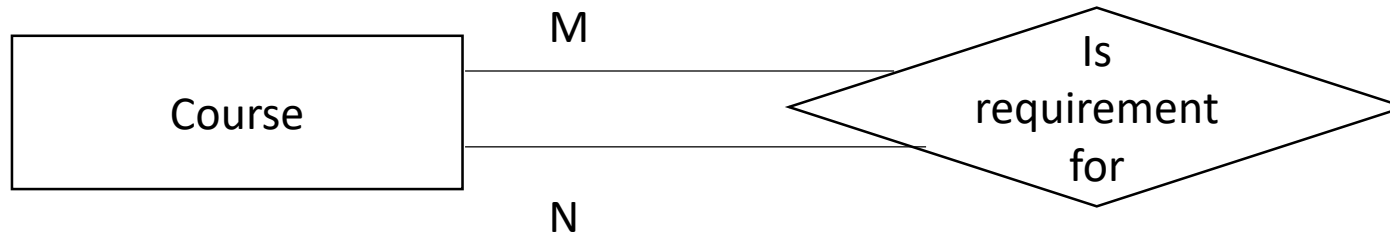
Unary Relationship

- **One to many (1:M) unary relationship:**



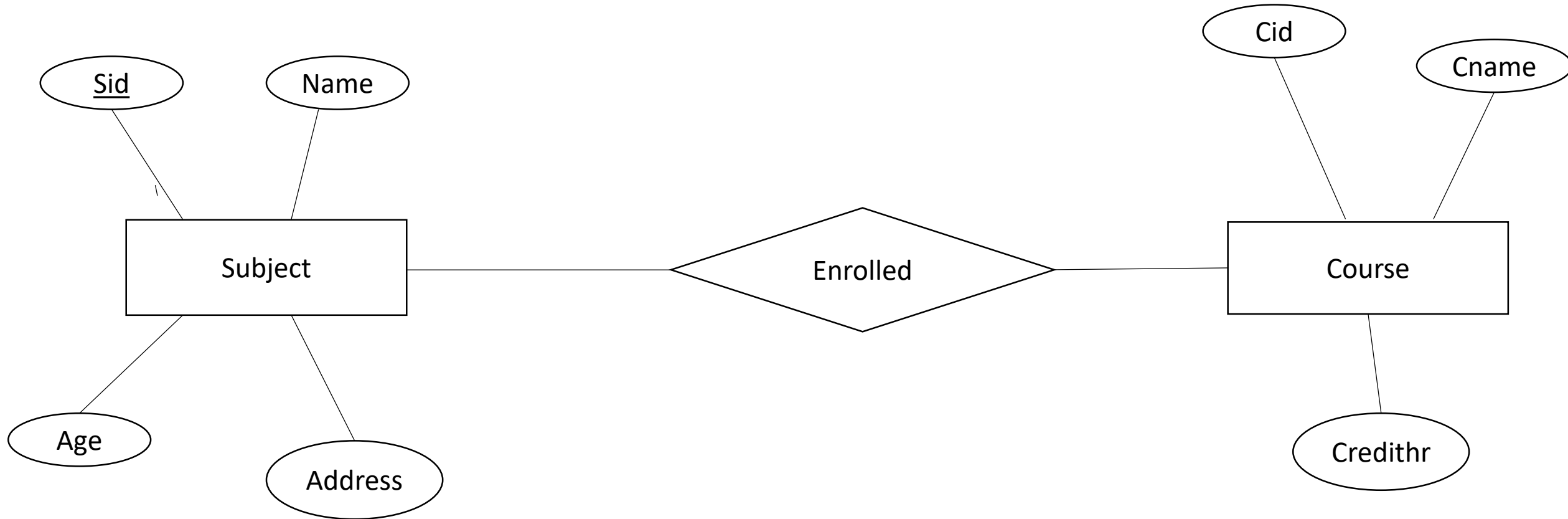
Unary Relationship

- **Many to many (M:N) unary relationship:**



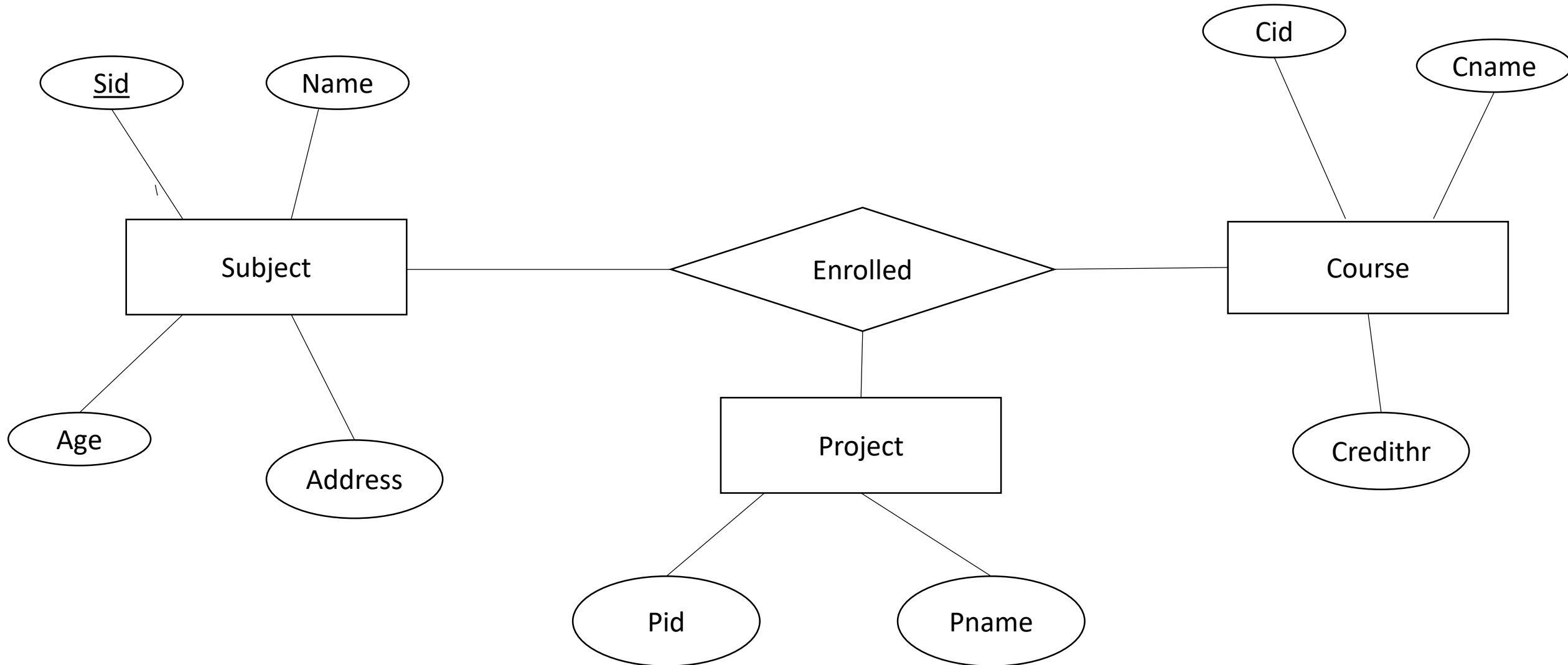
Binary Relationship

Two entity sets participate in a relationship.



N-ary Relationship

N entities set participating in a relation. (i.e. $n > 2$)



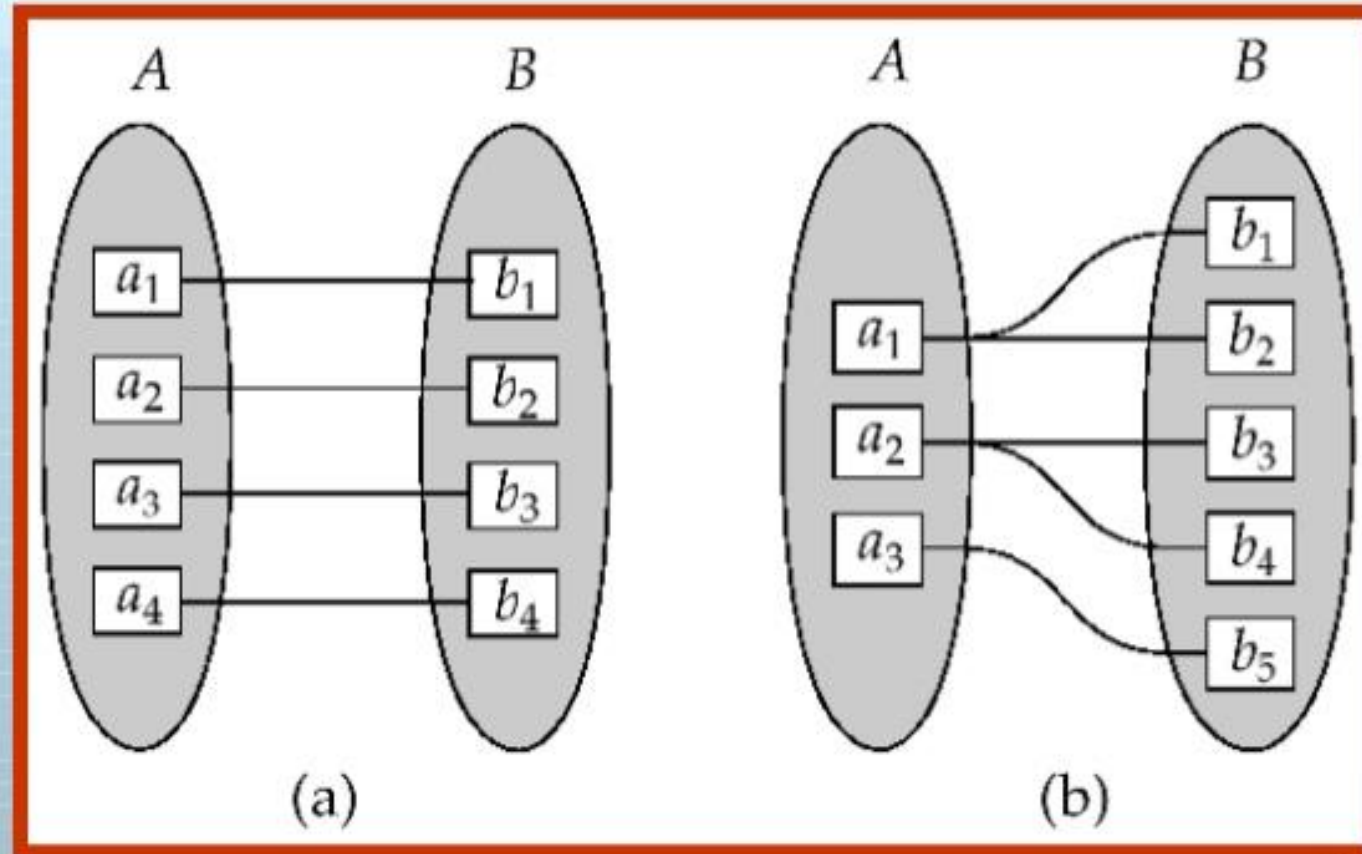


Mapping Cardinalities

- s Express the number of entities to which another entity can be associated via a relationship set.
- s Most useful in describing binary relationship sets.
- s For a binary relationship set the mapping cardinality must be one of the following types:
 - 5 One to one
 - 5 One to many
 - 5 Many to one
 - 5 Many to many



Mapping Cardinalities



One to one

One to many

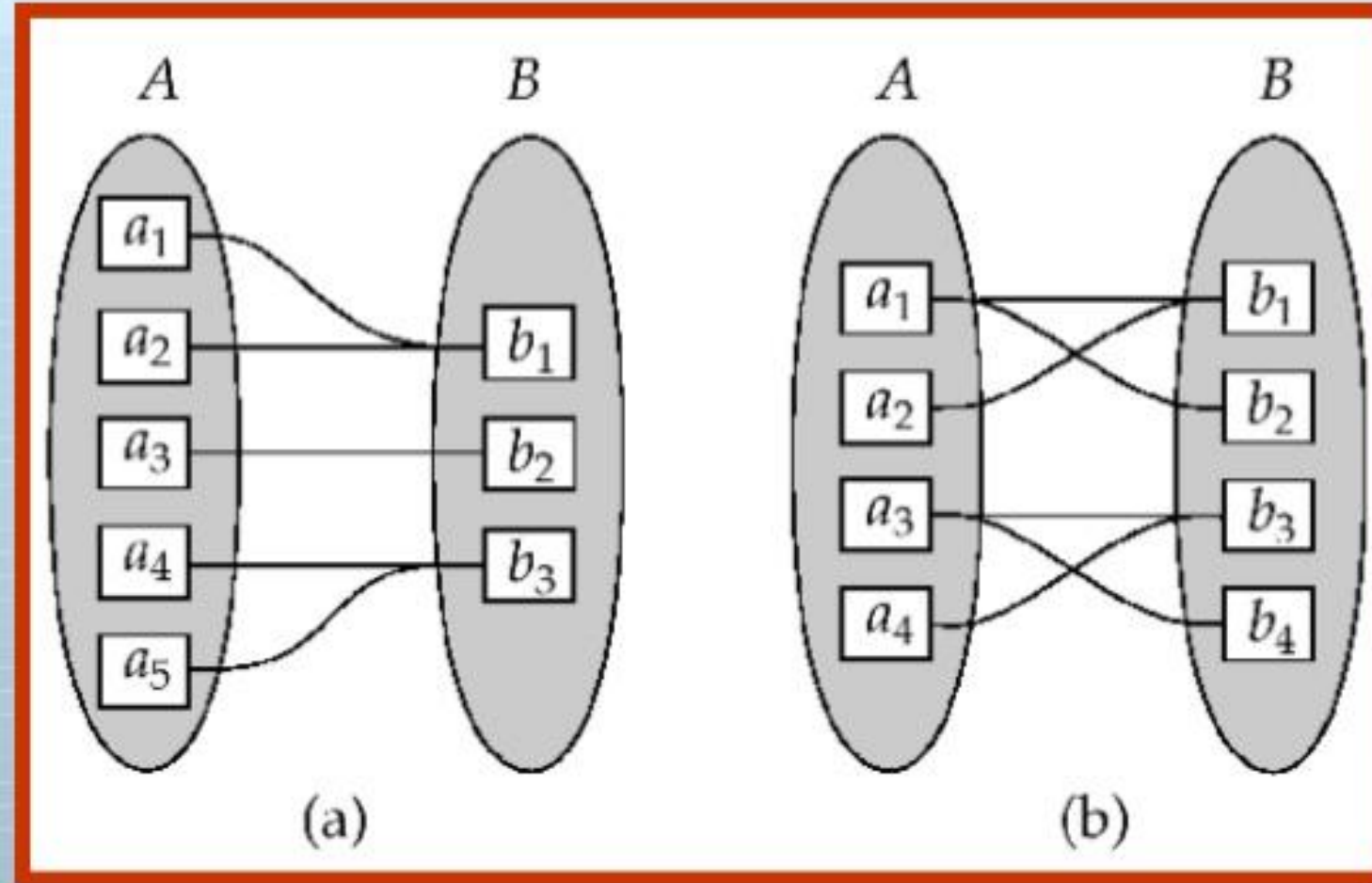
Note: Some elements in A and B may not be mapped to any elements in the other set

Pratiksha Poudel





Mapping Cardinalities



Many to one

Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set

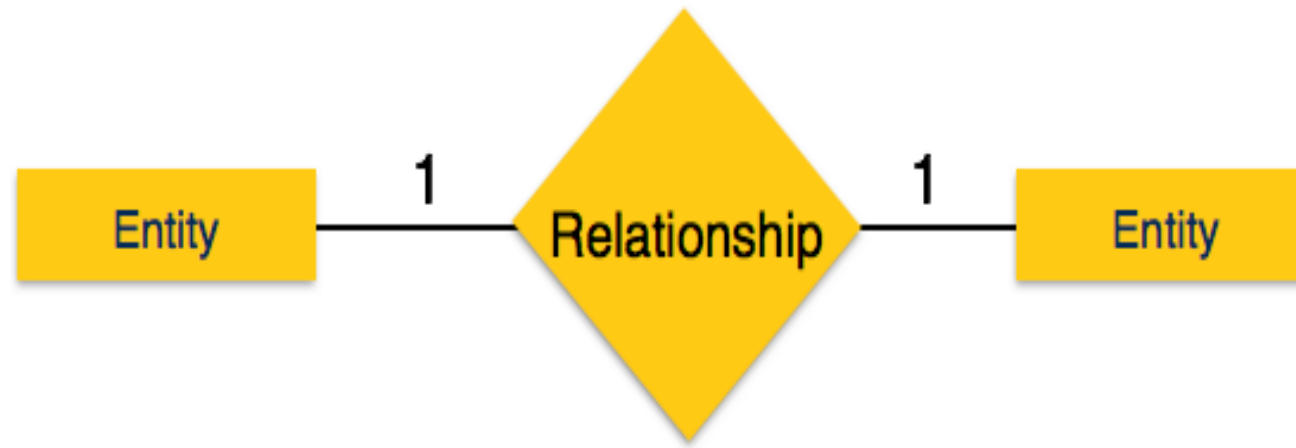
Relationship

- Relationships are represented by **diamond-shaped box**.
- Name of the relationship is written inside the diamond-box.
- All the entities (rectangles) participating in a relationship, are connected to it by a line.
- Binary Relationship and Cardinality
- A relationship where two entities are participating is called a **binary relationship**.

Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

One-to-one

When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport and a passport is given to one person.



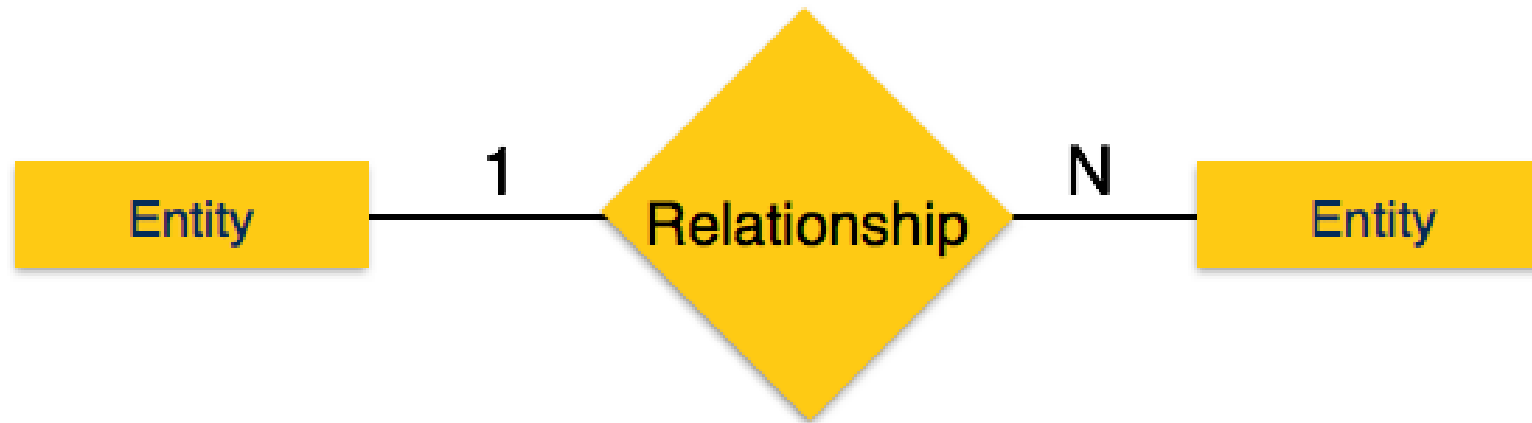
CARDINALITY CONSTRAINTS

- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line (—), signifying “many,” between the relationship set and the entity set.
- Or, by numbering each entity. * or, m for many.
- One-to-one relationship:
 - A student is associated with at most one *instructor* via the relationship *advisor*
 - A *student* is associated with at most one *department* via *stud_dept*



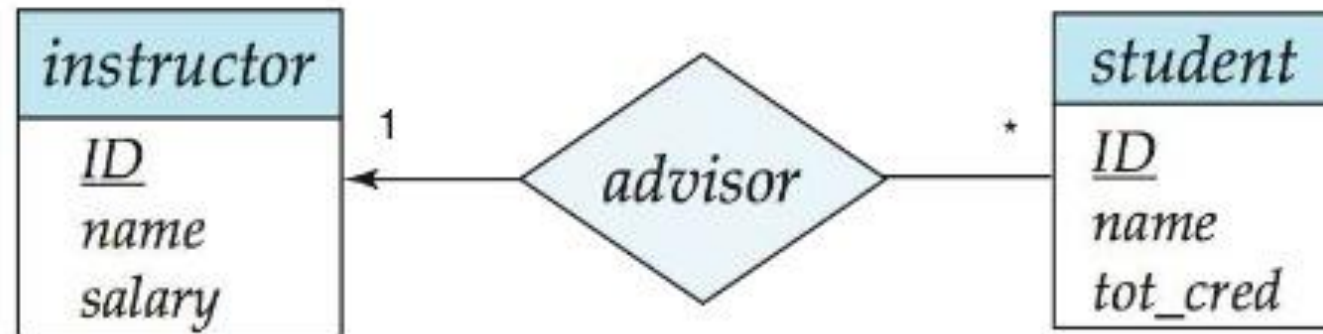
One-to-many

When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship. For example – a customer can place many orders but a order cannot be placed by many customers.



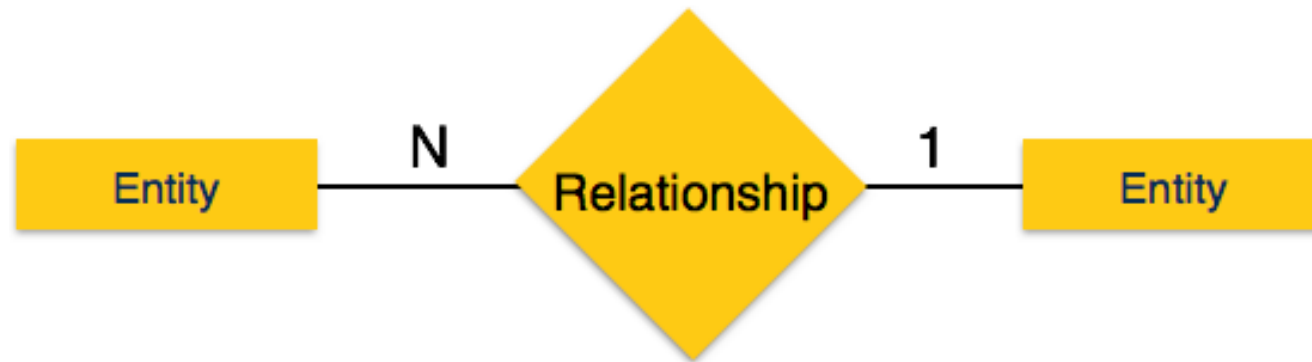
ONE-TO-MANY RELATIONSHIP

- one-to-many relationship between an *instructor* and a *student*
 - an instructor is associated with several (including 0) students via *advisor*
 - a student is associated with at most one instructor via *advisor*,



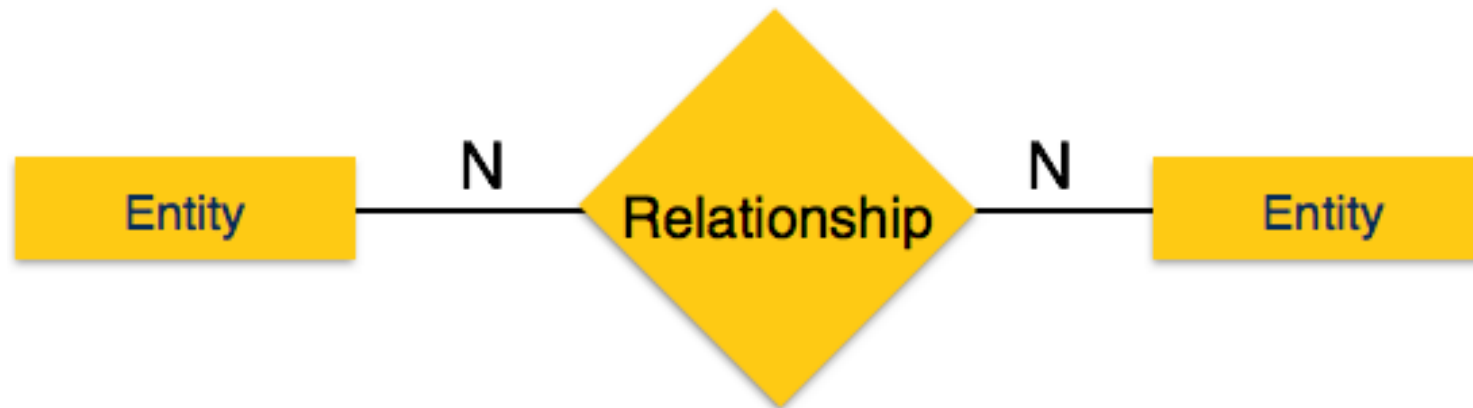
Many-to-one –

- When *more than one instance of entity is associated with the relationship*, it is marked as 'N:1'.
- When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.



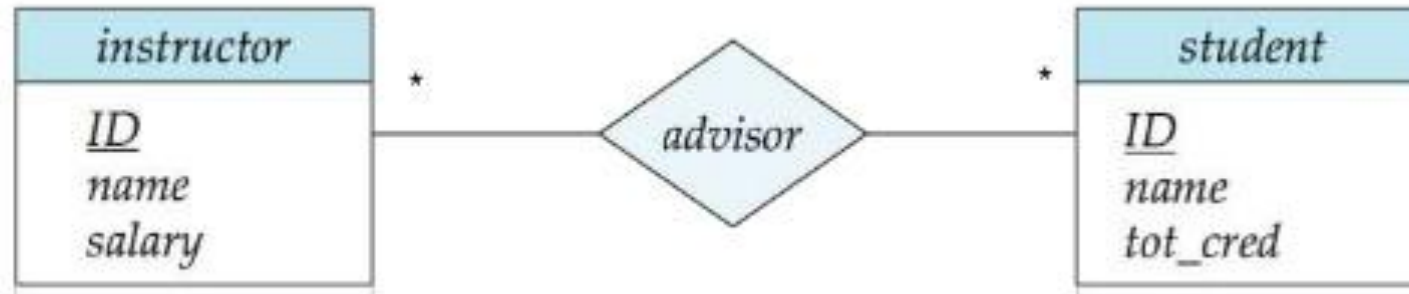
Many-to-many

When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship. For example, a can be assigned to many projects and a project can be assigned to many students. The following image reflects that *more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship*. It depicts many-to-many relationship.



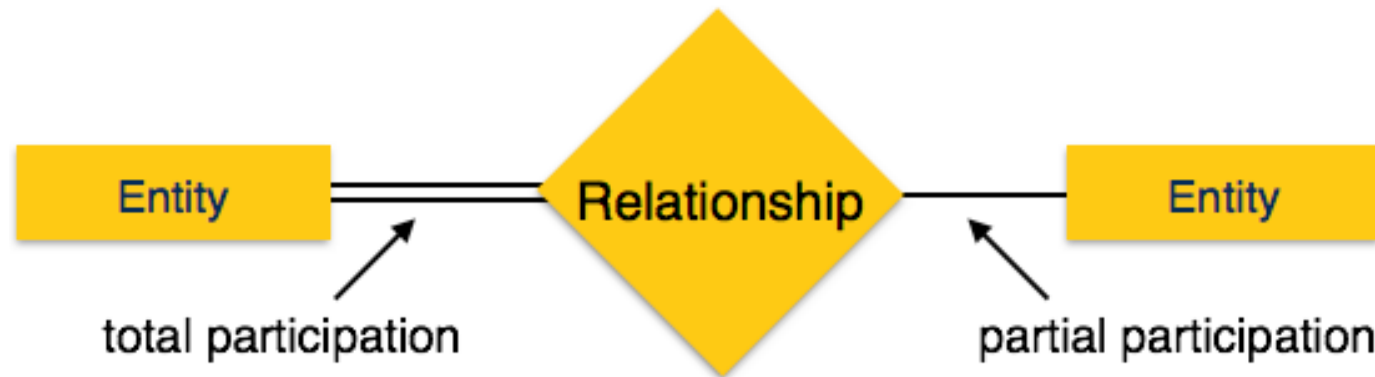
MANY-TO-MANY RELATIONSHIP

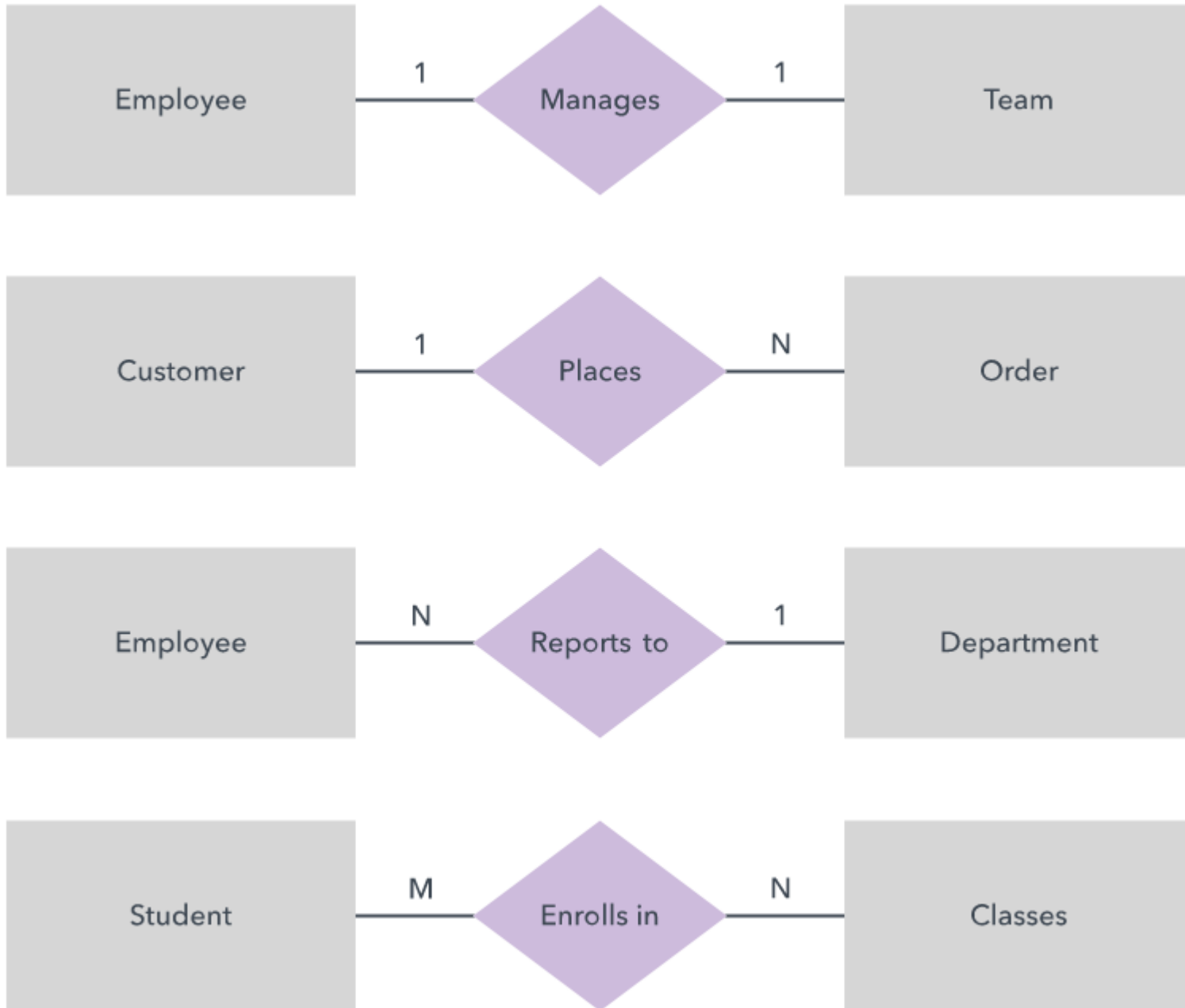
- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*



Participation Constraints

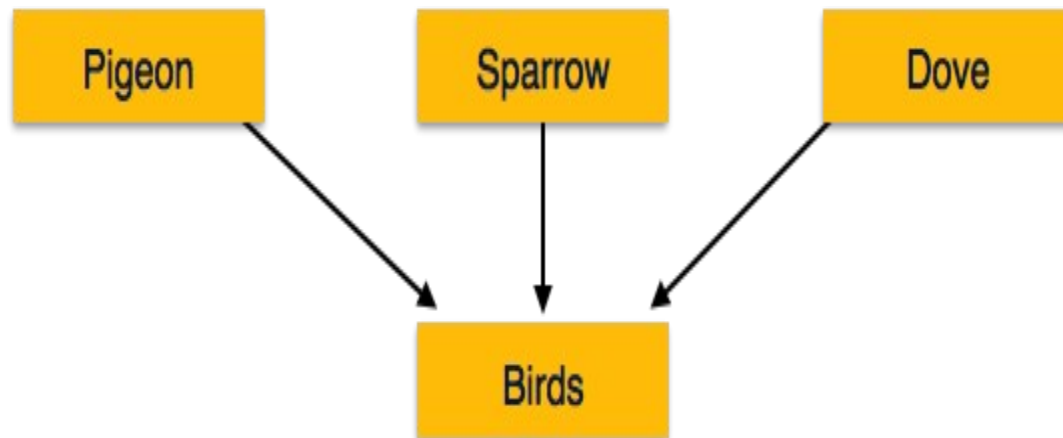
- **Total Participation** – Each entity is involved in the relationship. Total participation is represented by double lines.
- **Partial participation** – Not all entities are involved in the relationship. Partial participation is represented by single lines.





Generalization

- The process of generalizing entities, where the generalized entities contain the properties of all the generalized entities, is called generalization.
- In generalization, a number of entities are brought together into one generalized entity based on their similar characteristics. For example, pigeon, house sparrow, crow and dove can all be generalized as Birds.

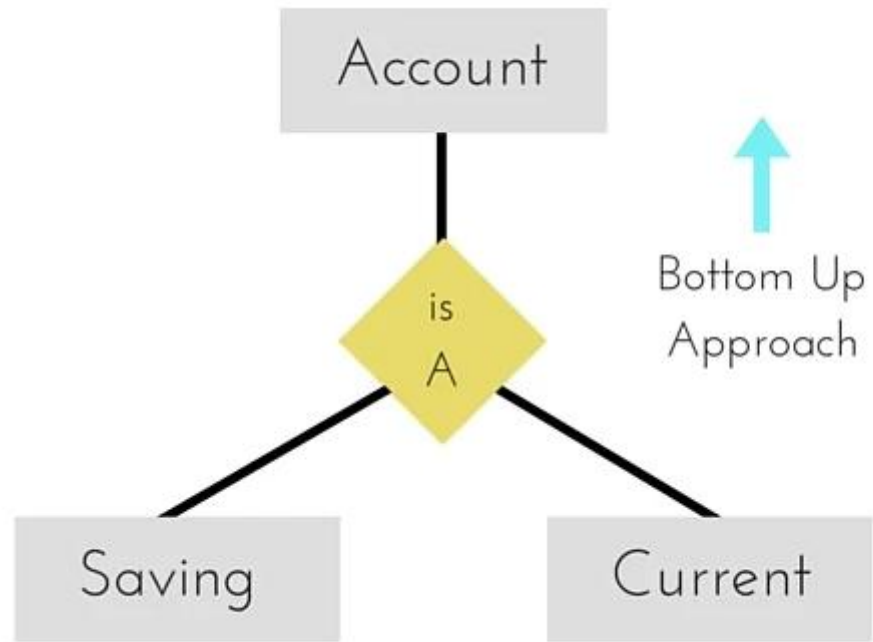


Generalization

- **Generalization** is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.
- It's more like Superclass and Subclass system, but the only difference is the approach, which is bottom-up. Hence, entities are combined to form a more generalised entity, in other words, sub-classes are combined to form a super-class.

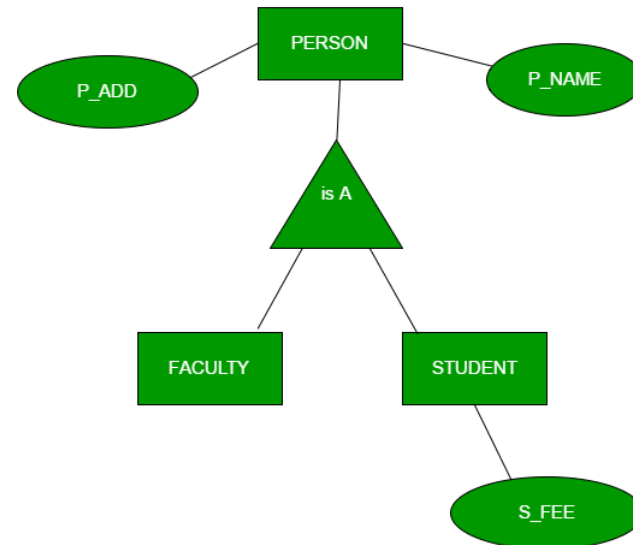
Generalization

- For example, **Saving** and **Current** account types entities can be generalised and an entity with name **Account** can be created, which covers both.



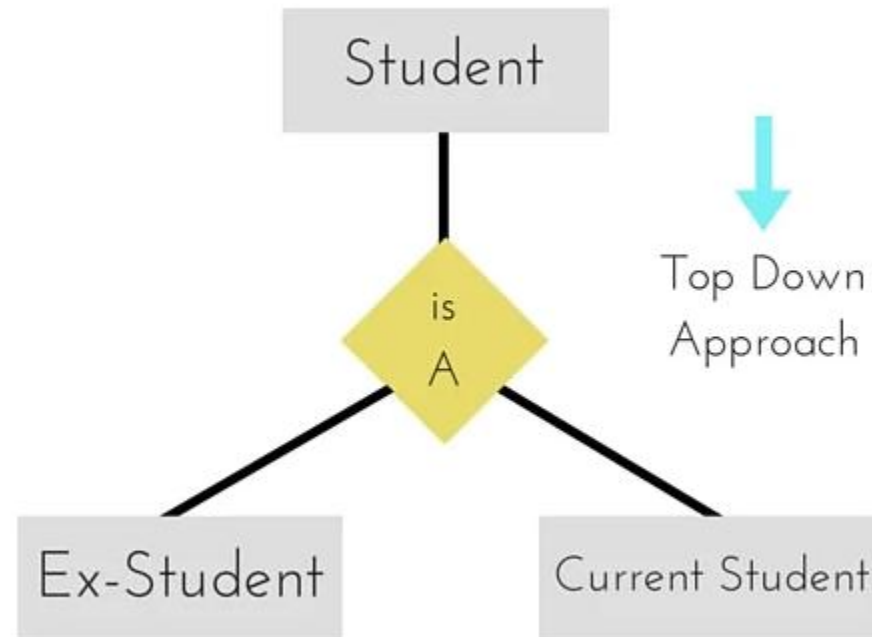
Generalization

Generalization is the process of extracting common properties from a set of entities and create a generalized entity from it. It is a bottom-up approach in which two or more entities can be generalized to a higher level entity if they have some attributes in common. For Example, STUDENT and FACULTY can be generalized to a higher level entity called PERSON as shown in Figure 1. In this case, common attributes like P_NAME, P_ADD become part of higher entity (PERSON) and specialized attributes like S_FEE become part of specialized entity (STUDENT).



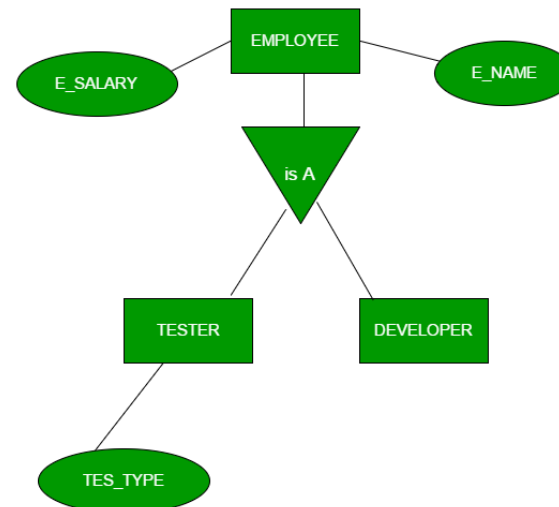
Specializations

- **Specialization** is opposite to Generalization. It is a top-down approach in which one higher level entity can be broken down into two lower level entity. In specialization, a higher level entity may not have any lower-level entity sets, it's possible.



Specializations

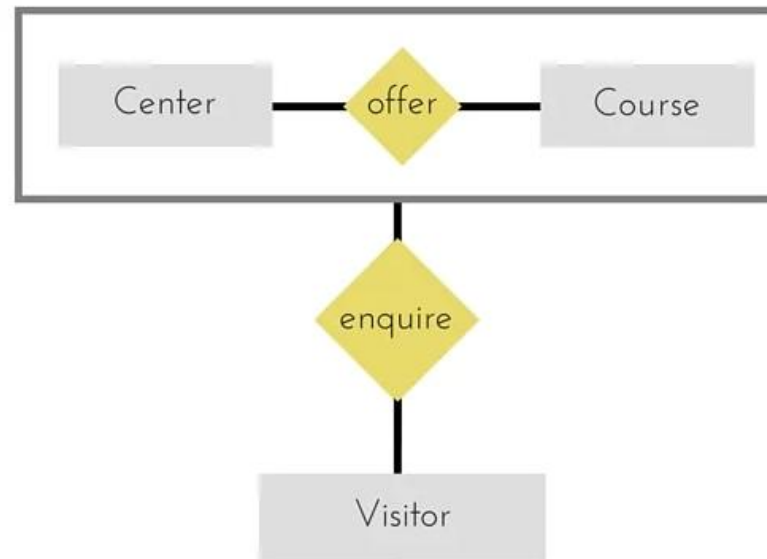
- In specialization, an entity is divided into sub-entities based on their characteristics. It is a top-down approach where higher level entity is specialized into two or more lower level entities. For Example, EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER etc. as shown in Figure 2. In this case, common attributes like E_NAME, E_SAL etc. become part of higher entity (EMPLOYEE) and specialized attributes like TES_TYPE become part of specialized entity (TESTER).



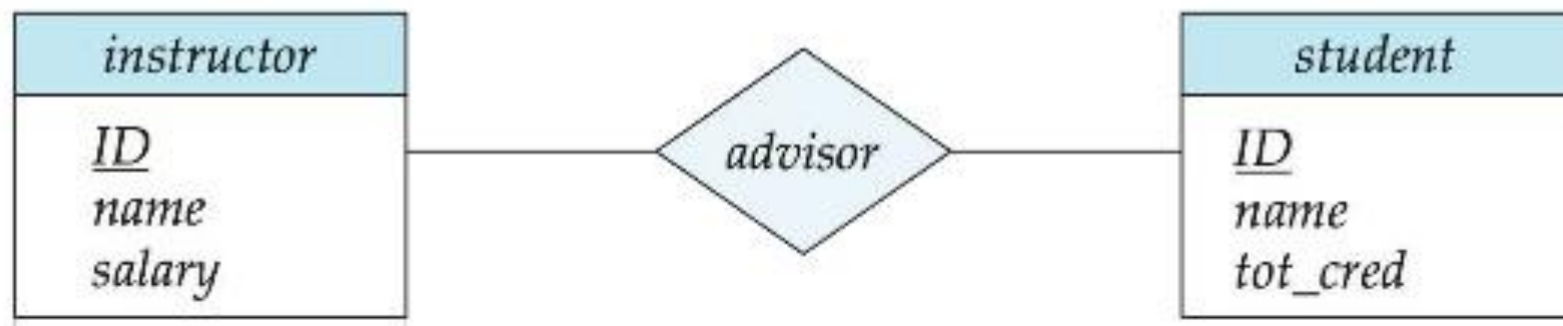
Specialization

Aggregation

- Aggregation is a process when relation between two entities is treated as a **single entity**.
- In the diagram above, the relationship between **Center** and **Course** together, is acting as an Entity, which is in relationship with another entity **Visitor**. Now in real world, if a Visitor or a Student visits a Coaching Center, he/she will never enquire about the center only or just about the course, rather he/she will ask enquire about both.



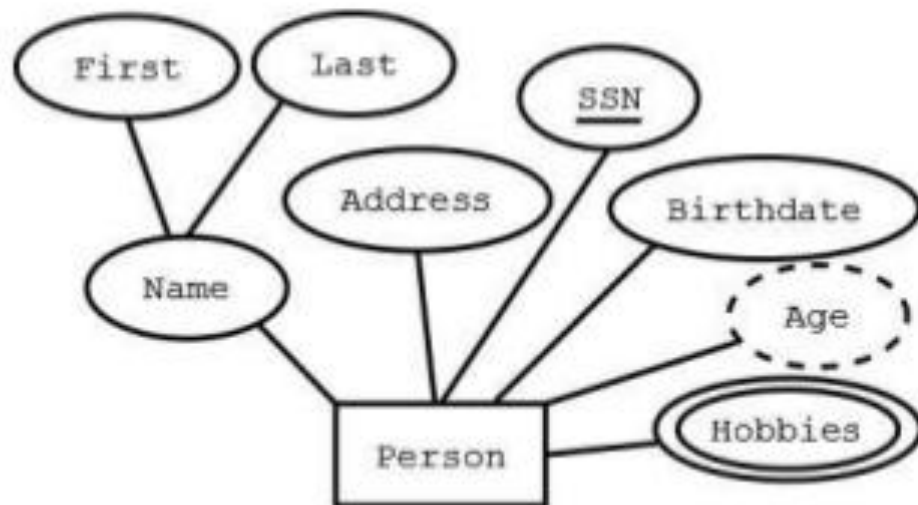
E-R DIAGRAMS



- Rectangles represent entity sets.
- Diamonds represent relationship sets.
- Attributes listed inside entity rectangle. Or , as oval shape along with the rectangle.
- Underline indicates primary key attributes

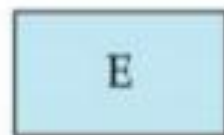
SUMMARY OF SYMBOLS USED IN E-R NOTATION

o Representing attributes



- **Rectangle** -- Entity
- **Ellipses** -- Attribute (underlined attributes are [part of] the primary key)
- **Double ellipses** -- multi-valued attribute
- **Dashed ellipses**-- derived attribute, e.g. age is derivable from birthdate and current date.

SUMMARY OF SYMBOLS USED IN E-R NOTATION



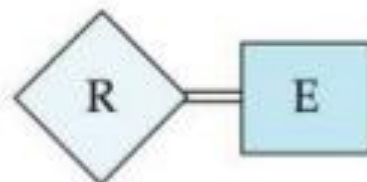
entity set



relationship set



identifying
relationship set
for weak entity set



total participation
of entity set in
relationship



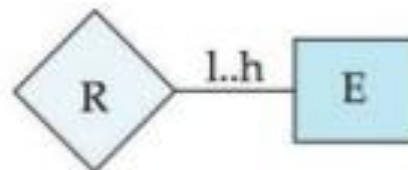
many-to-many
relationship



one-to-one
relationship



many-to-one
relationship



cardinality
limits

Creating ERD



Steps to Create an ER Diagram

1. Define the Entities.
2. Define the Relationships.
3. Add attributes to the relationships.
4. Add cardinality to the relationships.
5. Don't forget to use proper naming conventions and symbol representation.

Guidelines for Drawing ERDs

Example

In a university, a Student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a Professor can deliver only one course

Guidelines for Drawing ERDs

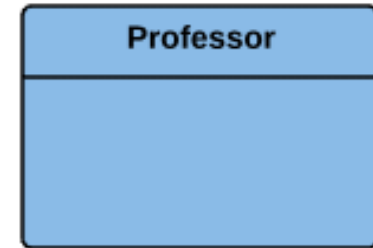
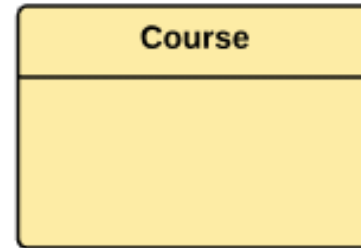
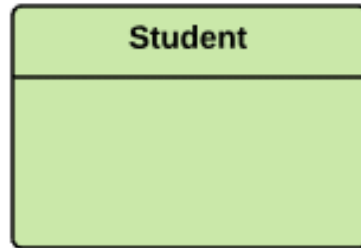
Step 1) Entity Identification

We have three entities

Student

Course

Professor



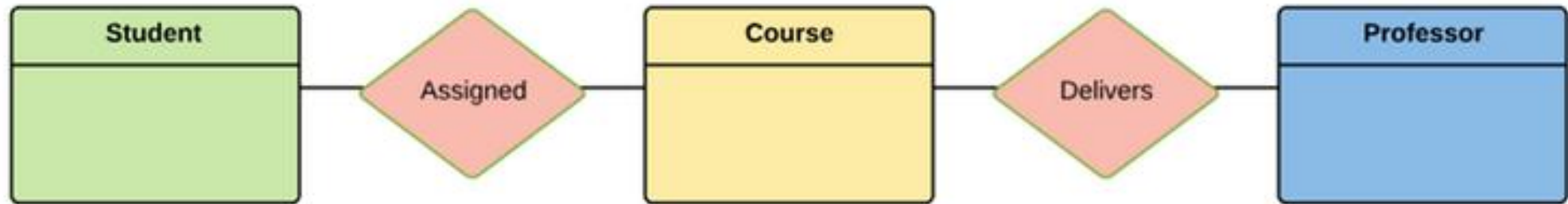
Guidelines for Drawing ERDs

Step 2) Relationship Identification

We have the following two relationships

The student is **assigned** a course

Professor **delivers** a course



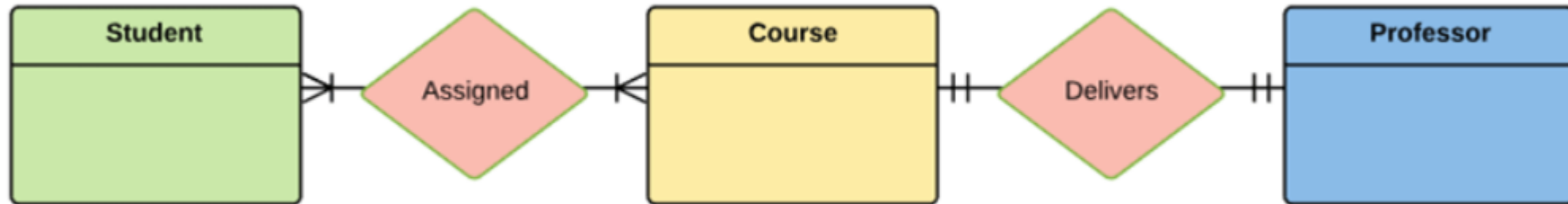
Guidelines for Drawing ERDs

Step 3) Cardinality Identification

For them problem statement we know that,

A student can be assigned **multiple** courses

A Professor can deliver only **one** course



Guidelines for Drawing ERDs

Step 4) Identify Attributes

You need to study the files, forms, reports, data currently maintained by the organization to identify attributes. You can also conduct interviews with various stakeholders to identify entities. Initially, it's important to identify the attributes without mapping them to a particular entity.

Once, you have a list of Attributes, you need to map them to the identified entities. Ensure an attribute is to be paired with exactly one entity. If you think an attribute should belong to more than one entity, use a modifier to make it unique.

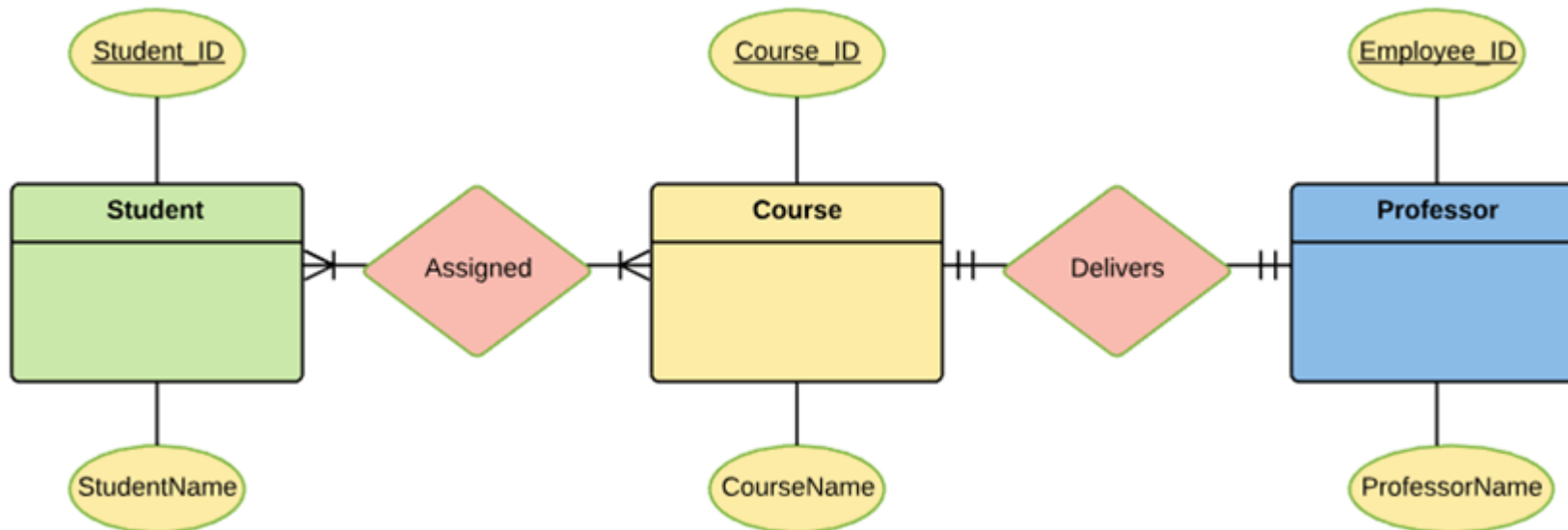
Guidelines for Drawing ERDs

Once the mapping is done, identify the primary Keys. If a unique key is not readily available, create one.

Entity	Primary Key	Attribute
Student	Student_ID	StudentName
Professor	Employee_ID	ProfessorName
Course	Course_ID	CourseName

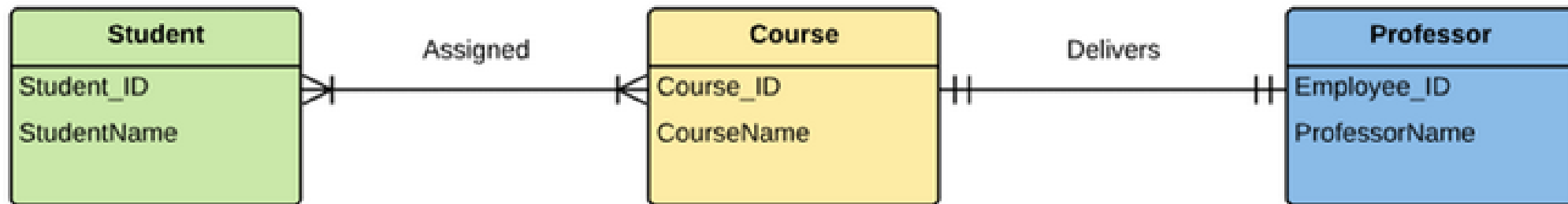
Guidelines for Drawing ERDs

For Course Entity, attributes could be Duration, Credits, Assignments, etc. For the sake of ease we have considered just one attribute.



Guidelines for Drawing ERDs

- **Step 5) Create the ERD Diagram**
- A more modern representation of Entity Relationship Diagram Example



Best Practices for Developing Effective ER Diagrams

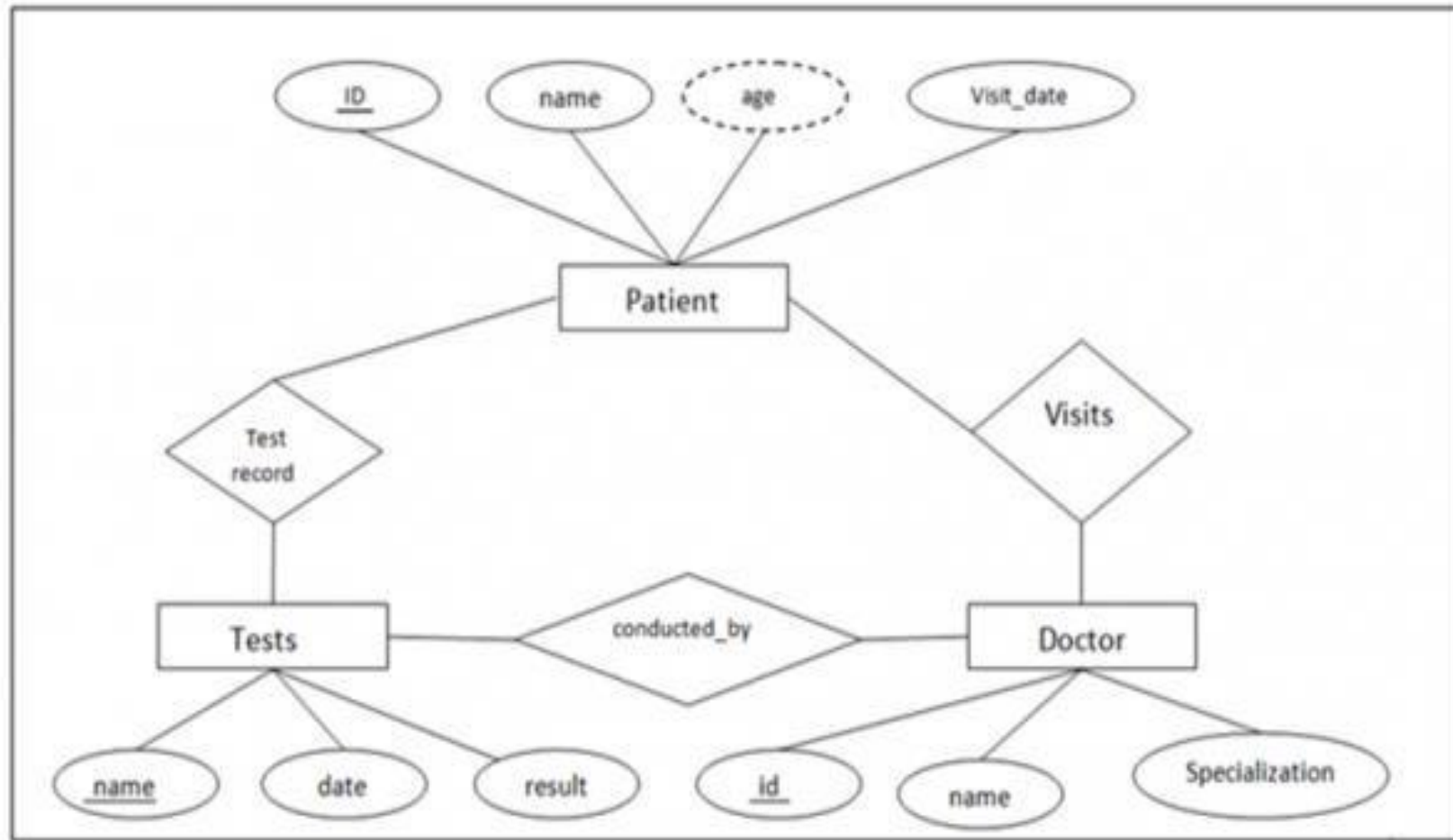
- Eliminate any redundant entities or relationships
- You need to make sure that all your entities and relationships are properly labeled
- There may be various valid approaches to an ER diagram. You need to make sure that the ER diagram supports all the data you need to store
- You should assure that each entity only appears a single time in the ER diagram
- Name every relationship, entity, and attribute are represented on your diagram
- Never connect relationships to each other
- You should use colors to highlight important portions of the ER diagram

Class Work

Draw ER Diagram of Hospital

- It has three entities: Patient, Doctor and Tests.
- Age is a derived attribute for Patient Entity
- Name in the Tests entity is a Primary Key
- ID in the Doctor entity is a Primary Key
- ID in the Patient entity is a Primary Key

Sample ER Diagram for Hospital



Class Work

Draw ER diagram of Library Management System

The Library Management System database keeps track of readers with the following considerations –

- The system keeps track of the staff with a single point authentication system comprising login Id and password.
- Staff maintains the book catalog with its ISBN, Book title, price(in INR), category(novel, general, story), edition, author Number and details.
- A publisher has publisher Id, Year when the book was published, and name of the book.
- Readers are registered with their user_id, email, name (first name, last name), Phone no (multiple entries allowed), communication address. The staff keeps track of readers.
- Readers can return/reserve books that stamps with issue date and return date. If not returned within the prescribed time period, it may have a due date too.
- Staff also generate reports that has readers id, registration no of report, book no and return/issue info.

Class Work

Draw ER diagram of Library Management System

Entities and their Attributes –

Book Entity : It has authno, isbn number, title, edition, category, price. ISBN is the Primary Key for Book Entity.

Reader Entity : It has UserId, Email, address, phone no, name. Name is composite attribute of firstname and lastname. Phone no is multi valued attribute. UserId is the Primary Key for Readers entity.

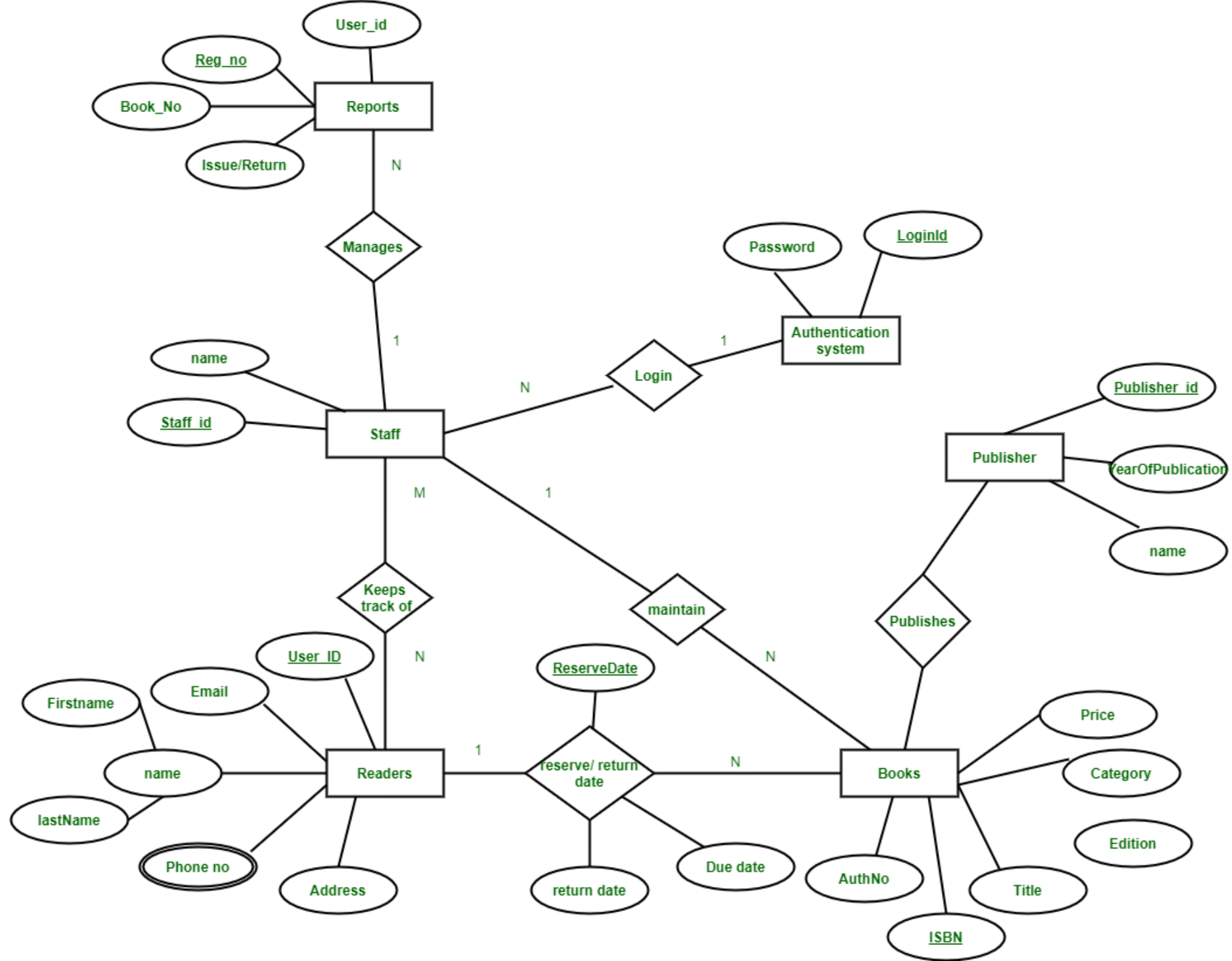
Publisher Entity : It has PublisherId, Year of publication, name. PublisherID is the Primary Key.

Authentication System Entity : It has LoginId and password with LoginID as Primary Key.

Reports Entity : It has UserId, Reg_no, Book_no, Issue/Return date. Reg_no is the Primary Key of reports entity.

Staff Entity : It has name and staff_id with staff_id as Primary Key.

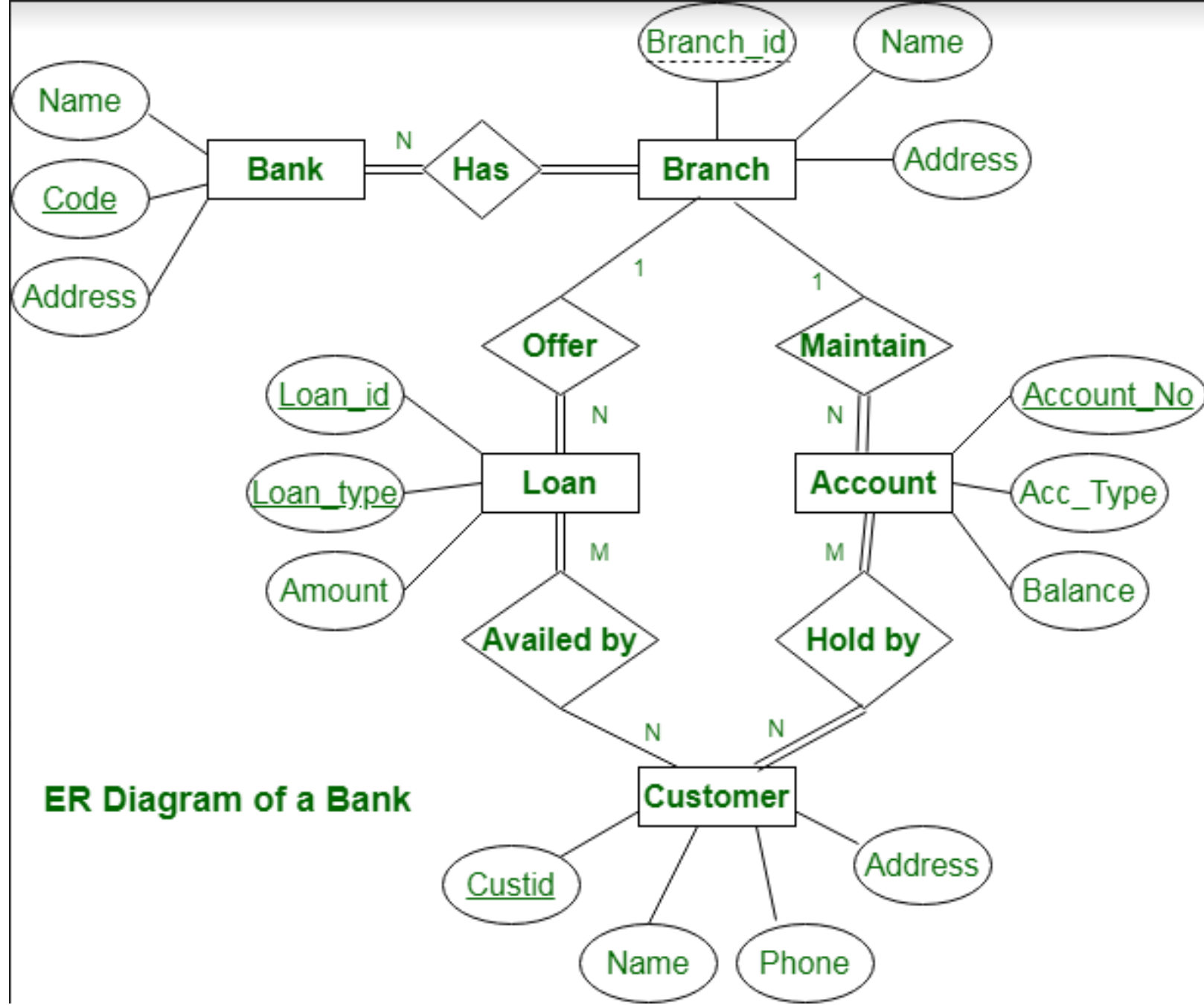
Reserve/Return Relationship Set : It has three attributes: Reserve date, Due date, Return date.



ER diagram of Bank Management System

ER diagram of Bank has the following description :

- Bank have Customer.
- Banks are identified by a name, code, address of main office.
- Banks have branches.
- Branches are identified by a branch_no., branch_name, address.
- Customers are identified by name, cust-id, phone number, address.
- Customer can have one or more accounts.
- Accounts are identified by acc_no., acc_type, balance.
- Customer can avail loans.
- Loans are identified by loan_id, loan_type and amount.
- Account and loans are related to bank's branch.



Class Work

Draw ER Diagram of Online Exam System

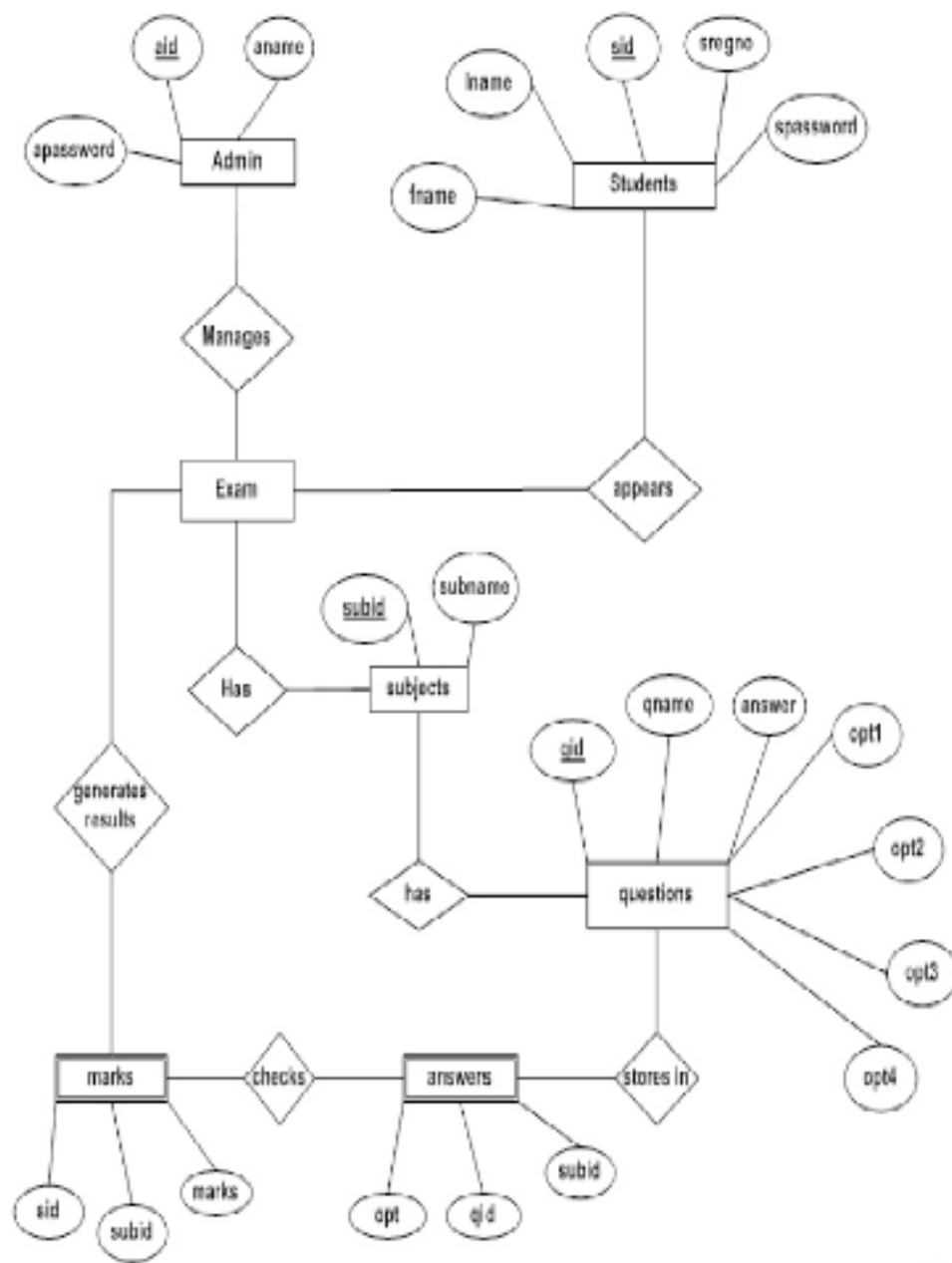
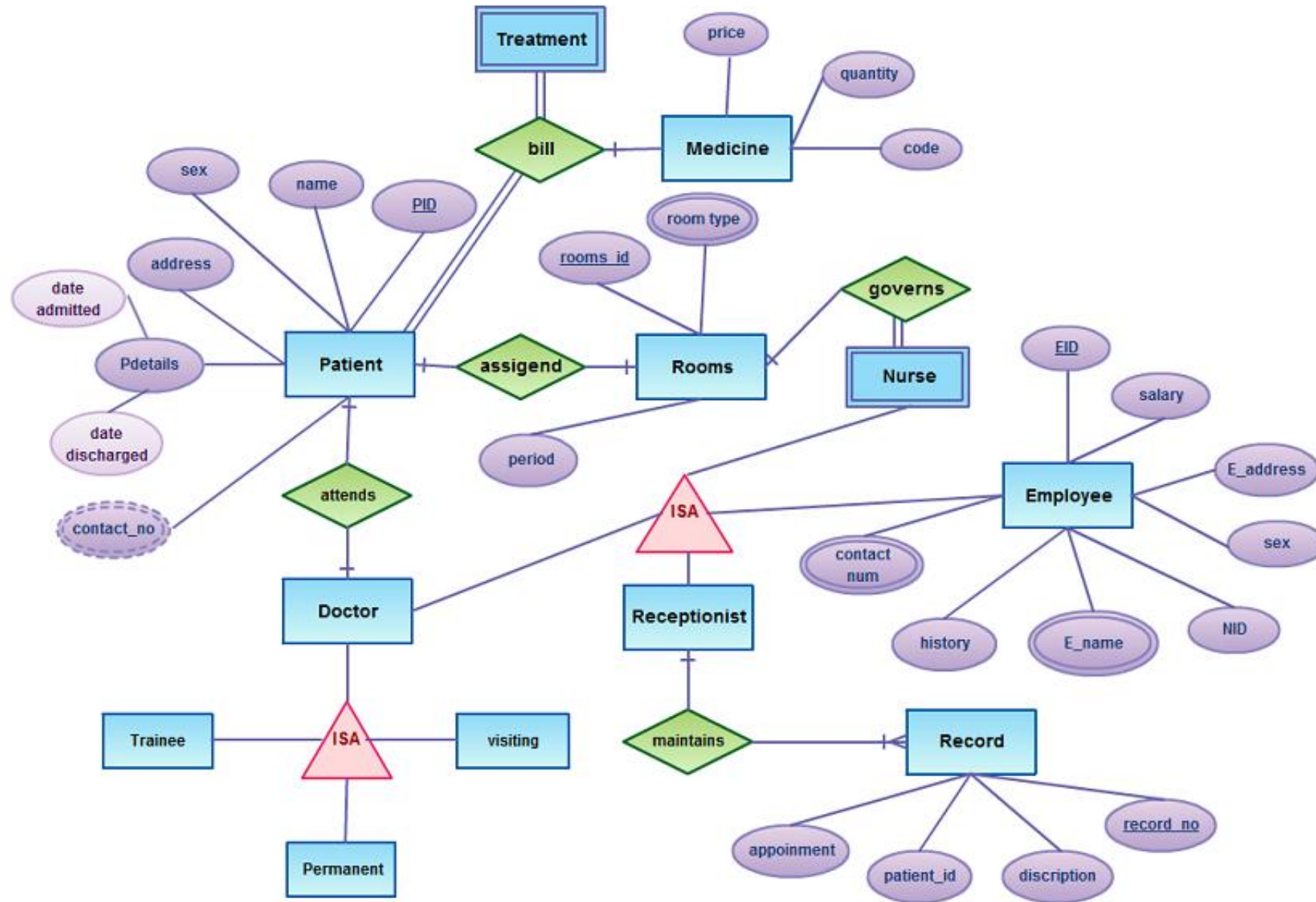


Fig 9.1:- ER diagram for online examination system.

Class Work

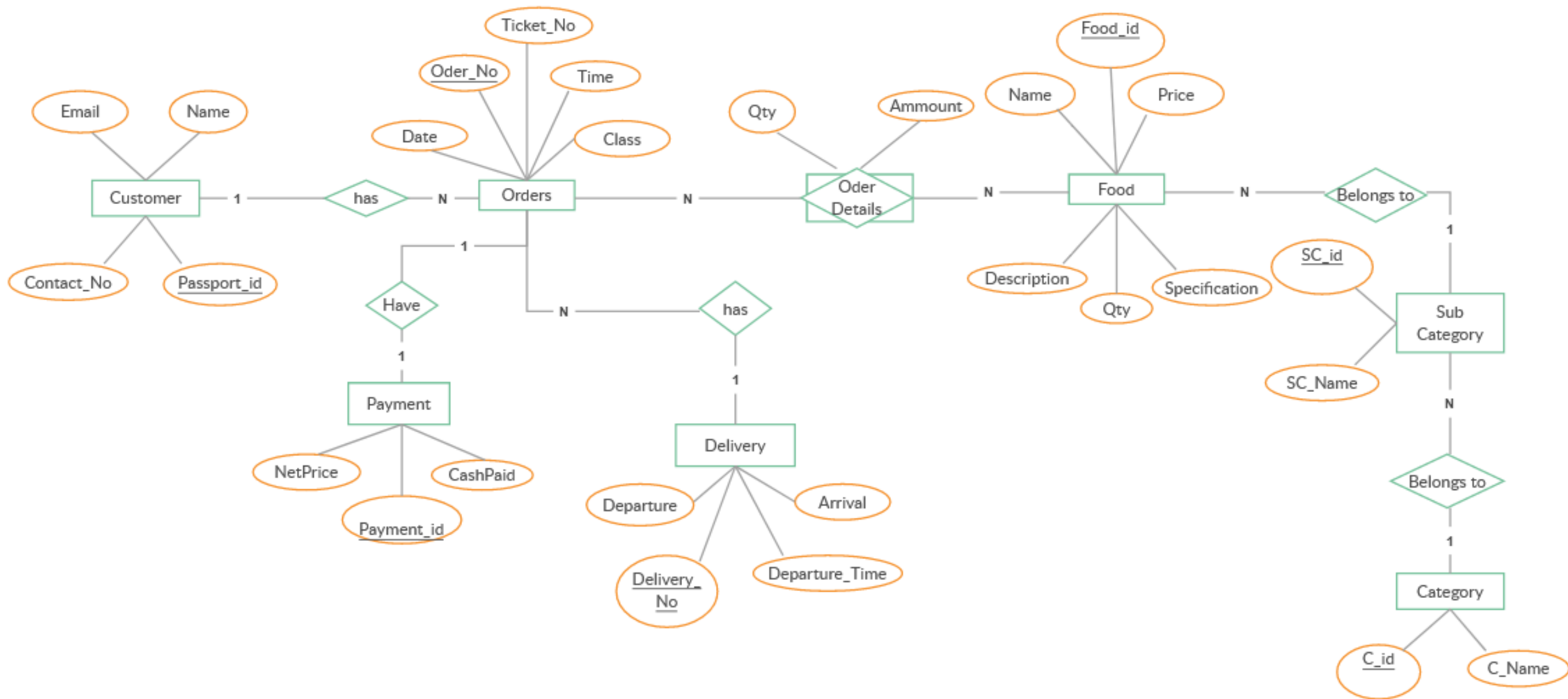
Draw ER Diagram of Hospital Management System

E-R Diagram for Hospital Management System



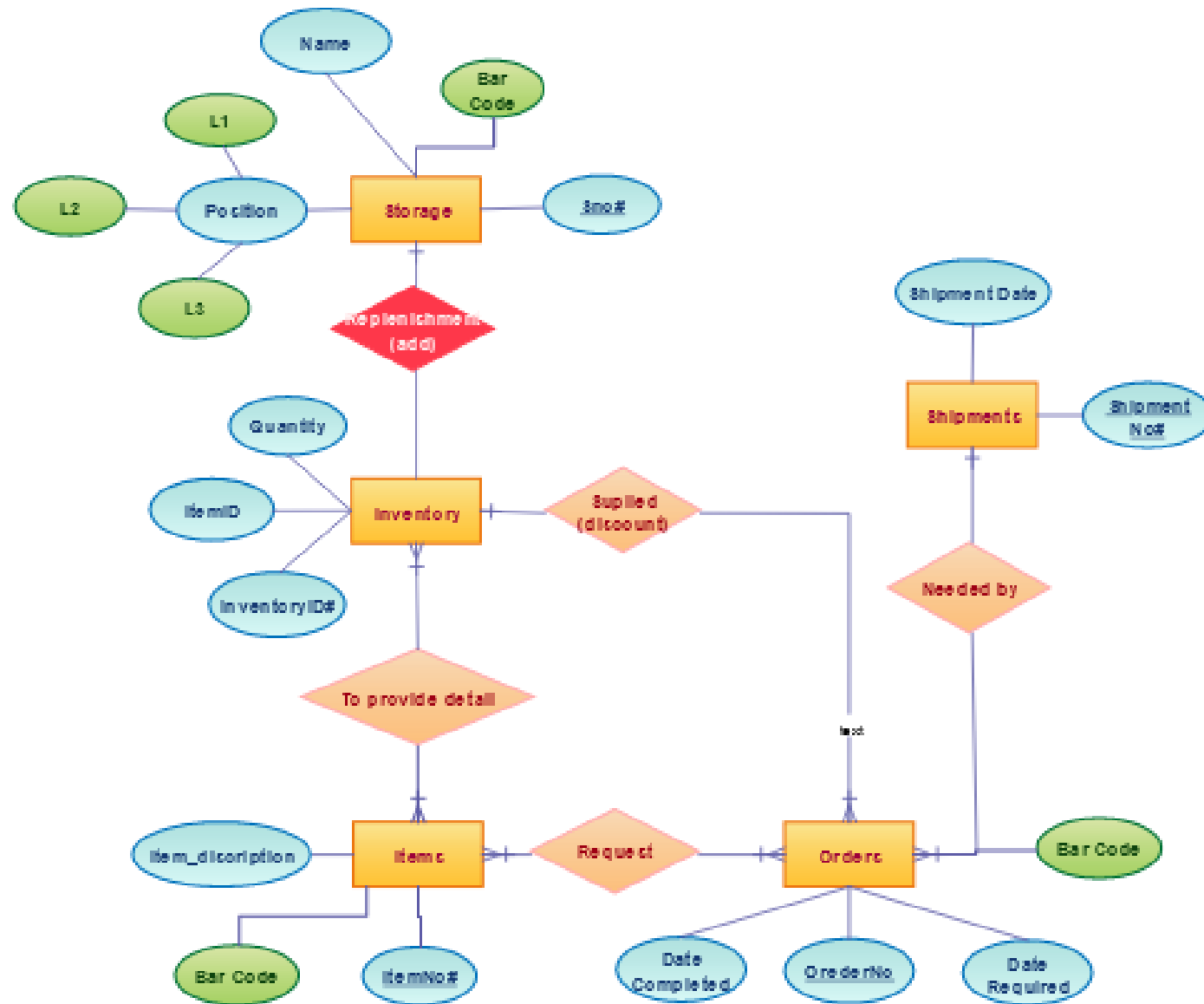
Class Work

Draw ER Diagram of Food Ordering System



Class Work

Draw ER Diagram of Inventory Management System



Assignment

Draw ER Diagram & DFD (Level 0, Level 1 & Level 2) of

- a. University MIS
- b. Hotel Management System
- c. Online Exam Form
- d. Online Flight ticket Reservation System
- e. E-voting System

THANK YOU!